

Project work: in-depth study of cosine similarity–based loss functions for face recognition tasks: AdaCos, ArcFace, CosFace.

Face Recognition and Angular Margin Loss

The task of face recognition is to learn an embedding representation of a face such that images of the same person are close in feature space, while images of different people are far apart.

This is usually done by training a model with classification, giving a geometric structure to the embedding space, which is often normalized.

Here, classification is not just a linear separation problem, but an **angular separation problem**. Each class has a weight vector W_i , and the prediction depends on the angle θ_i between the image embedding x and the class vector.

The logit for class i is based on cosine similarity:

$$\cos \theta_i = W_i^T x$$

Angular Margin Loss

To make embeddings more discriminative, **angular margin losses** were introduced. These losses force a bigger separation between the correct class and other classes.

Two common methods are **CosFace** and **ArcFace**, which differ in how they apply the margin.

CosFace: Margin on Cosine

Let x be the feature vector and w_j the class weight vectors, both **L2-normalized**.

The logit of class j is defined as

$$z_j = s \cos \theta_j = s w_j^T x,$$

where θ_j is the angle between x and w_j , and s is a scale factor.

CosFace introduces an additive margin in the **cosine space** by modifying only the logit of the correct class y :

$$z_y = s(\cos \theta_y - m), z_j = s \cos \theta_j (j \neq y).$$

The decision condition between the correct class y and another class j becomes

$$\cos \theta_y - m > \cos \theta_j,$$

which means that the cosine similarity with the correct class center must be larger than that of any other class by at least m .

Since the margin is applied directly to $\cos \theta$, CosFace does **not** produce a constant angular margin.

Let us define an equivalent angle $\theta^{\hat{}}$ such that

$$\cos \theta^{\hat{}} = \cos \theta - m.$$

Then

$$\theta^{\hat{}} = \arccos(\cos \theta - m), \Delta \theta = \theta^{\hat{}} - \theta.$$

The effective angular margin induced by CosFace is therefore

$$\Delta \theta(\theta) = \arccos(\cos \theta - m) - \theta,$$

which explicitly depends on the angle θ .

It's also possible to define another approximated relation to better understand how the angular margin is applied:

Starting from the definition of CosFace:

$$\cos \theta \rightarrow \cos \theta - m.$$

This means that the new value of the cosine is

$$(\cos \theta)_{\text{new}} = (\cos \theta)_{\text{old}} - m.$$

Therefore, the change in the cosine is

$$\Delta(\cos \theta) = (\cos \theta)_{\text{new}} - (\cos \theta)_{\text{old}}.$$

Substituting,

$$\Delta(\cos \theta) = (\cos \theta - m) - \cos \theta = -m.$$

So we have

$$\Delta(\cos \theta) = -m.$$

Now, we connect this to the differential formula. We know that

$$d(\cos \theta) = -\sin \theta d\theta.$$

If we assume small changes in the neighborhood of θ , we can approximate

$$\Delta(\cos \theta) \approx -\sin \theta \Delta \theta.$$

Substituting the change imposed by CosFace,

$$-m \approx -\sin \theta \Delta \theta.$$

Therefore,

$$\Delta \theta \approx \frac{m}{\sin \theta}.$$

This shows that the angular margin induced by CosFace depends on the position of the sample on the hypersphere (that is, on θ) and is therefore **not uniform in the angular space**.

ArcFace: Margin on Angle

ArcFace removes this ambiguity. It adds the margin directly to the angle of the correct class:

$$\cos(\theta_i + m)$$

The decision boundary becomes:

$$\cos(\theta_i + m) = \cos \theta_j$$

The angular separation is now constant, independent of the embedding position.

Geometric interpretation

To visually understand the difference in how the margin is applied in **ArcFace** and **CosFace**, it is useful to look at the shape of the function

- **CosFace (vertical shift)**
 - penalizes the cosine value y ,
 - the induced angular difficulty depends on the local slope of $\cos(x)$.
 - **ArcFace (horizontal shift)**
 - penalizes the input angle x ,
 - enforces a constant and well-defined angular margin.
-

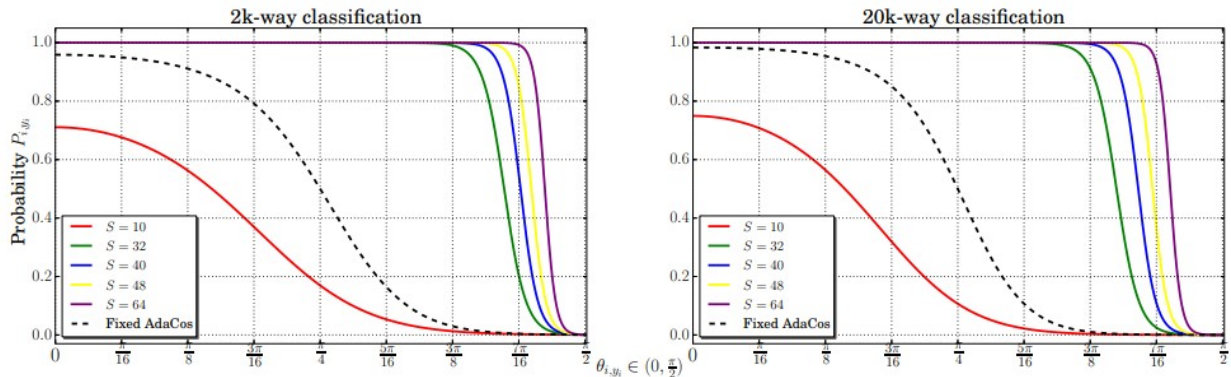
Adacos

Both **CosFace** and **ArcFace** require careful manual tuning of hyperparameters such as the angular margin and the scaling factor s , which strongly influence training and final performance. Choosing a non-optimal value of s can cause two main problems:

- If s is too large, the probability curve **saturates quickly**, so the model reaches very high logit values early and the softmax function becomes almost flat, reducing useful gradient

information.

- If s is too small, predicted probabilities remain low even for correct classes, and the model **over-penalizes predictions that are already qualitatively correct**, slowing convergence and reducing the effectiveness of supervision.



To overcome these issues, in this analysis I used **AdaCos**, a loss that introduces **adaptive scaling** of the logits without the usage of the margin. AdaCos automatically computes the scale factor based on the distribution of angles between embeddings and class vectors during training. This allows the model to **progressively adjust the supervision strength**, eliminating the need for manual tuning.

The dynamic scale parameter for the t -th iteration is formulated as:

$$\tilde{s}_d^{(t)} = \begin{cases} \sqrt{2 \cdot \log(C-1)}, & t=0, \\ \frac{\log B_{avg}^{(t)}}{\cos(\min(\pi/4, \theta_{med}^{(t)}))}, & t \geq 1 \end{cases}$$

Here:

- C is the number of classes,
- $B_{avg}^{(t)}$ is the average of the batch logits,
- $\theta_{med}^{(t)}$ is the median angle of the current mini-batch.

```
# @title useful imports
import os
import math
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from PIL import Image
import torch
import torch.nn as nn
import torch.nn.functional as F
```

```

import torch.optim as optim
from torch.nn import Parameter
from torch.utils.data import DataLoader
from torchvision import transforms
import timm
from pytorch_metric_learning import losses
from sklearn.neighbors import KNeighborsClassifier
from tqdm.auto import tqdm

# @title useful functions
import gc
def clean_gpu_memory(model=None):
    """
    Pulisce la cache della GPU e cancella il modello se passato.
    """
    if model is not None:
        del model

    gc.collect()
    if torch.cuda.is_available():
        torch.cuda.empty_cache()

    print(">>> GPU Memory Cleaned.")

def load_checkpoint(model, filepath, device='cuda'):
    """
    Carica i pesi gestendo 'module.' prefix e map_location.
    """
    if not os.path.exists(filepath):
        print(f"ERRORE: Il file {filepath} non esiste.")
        return model

    print(f"Caricamento pesi da: {filepath}...")

    # Carica il dizionario dei pesi
    checkpoint = torch.load(filepath, map_location=device)

    # Se il checkpoint è salvato come dict intero (es:
    # {'model_state_dict': ...})
    if 'model_state_dict' in checkpoint:
        state_dict = checkpoint['model_state_dict']
    else:
        state_dict = checkpoint

    # Rimuovi il prefisso "module." se il modello è stato trainato in
    DataParallel
    new_state_dict = {}
    for k, v in state_dict.items():
        name = k.replace("module.", "")
        new_state_dict[name] = v

```

```

# Carica nel modello
try:
    model.load_state_dict(new_state_dict, strict=True)
    print(">>> Pesi caricati con successo (Strict=True).")
except RuntimeError as e:
    print(f"Warning: strict loading failed, retrying with
strict=False. Error: {e}")
    model.load_state_dict(new_state_dict, strict=False)
    print(">>> Pesi caricati (Strict=False).")

model.to(device)
model.eval()
return model

!pip install pytorch-metric-learning

pip install deepface

Collecting deepface
  Downloading deepface-0.0.98-py3-none-any.whl.metadata (33 kB)
Requirement already satisfied: requests>=2.27.1 in
/usr/local/lib/python3.12/dist-packages (from deepface) (2.32.5)
Requirement already satisfied: numpy>=1.14.0 in
/usr/local/lib/python3.12/dist-packages (from deepface) (2.0.2)
Requirement already satisfied: pandas>=0.23.4 in
/usr/local/lib/python3.12/dist-packages (from deepface) (2.2.2)
Requirement already satisfied: gdown>=3.10.1 in
/usr/local/lib/python3.12/dist-packages (from deepface) (5.2.0)
Requirement already satisfied: tqdm>=4.30.0 in
/usr/local/lib/python3.12/dist-packages (from deepface) (4.67.1)
Requirement already satisfied: Pillow>=5.2.0 in
/usr/local/lib/python3.12/dist-packages (from deepface) (11.3.0)
Requirement already satisfied: opencv-python>=4.5.5.64 in
/usr/local/lib/python3.12/dist-packages (from deepface) (4.12.0.88)
Requirement already satisfied: tensorflow>=1.9.0 in
/usr/local/lib/python3.12/dist-packages (from deepface) (2.19.0)
Requirement already satisfied: keras>=2.2.0 in
/usr/local/lib/python3.12/dist-packages (from deepface) (3.10.0)
Requirement already satisfied: Flask>=1.1.2 in
/usr/local/lib/python3.12/dist-packages (from deepface) (3.1.2)
Collecting flask-cors>=4.0.1 (from deepface)
  Downloading flask_cors-6.0.2-py3-none-any.whl.metadata (5.3 kB)
Collecting mtcnn>=0.1.0 (from deepface)
  Downloading mtcnn-1.0.0-py3-none-any.whl.metadata (5.8 kB)
Collecting retina-face>=0.0.14 (from deepface)
  Downloading retina_face-0.0.17-py3-none-any.whl.metadata (10 kB)
Collecting fire>=0.4.0 (from deepface)
  Downloading fire-0.7.1-py3-none-any.whl.metadata (5.8 kB)
Collecting unicorn>=20.1.0 (from deepface)

```

Downloading gunicorn-25.0.0-py3-none-any.whl.metadata (4.7 kB)
Collecting lightphe>=0.0.15 (from deepface)
Downloading lightphe-0.0.20-py3-none-any.whl.metadata (13 kB)
Collecting lightds>=0.0.3 (from deepface)
Downloading lightds-0.0.3-py3-none-any.whl.metadata (7.3 kB)
Requirement already satisfied: python-dotenv>=1.0.1 in
/usr/local/lib/python3.12/dist-packages (from deepface) (1.1.1)
Requirement already satisfied: termcolor in
/usr/local/lib/python3.12/dist-packages (from fire>=0.4.0->deepface)
(3.1.0)
Requirement already satisfied: blinker>=1.9.0 in
/usr/local/lib/python3.12/dist-packages (from Flask>=1.1.2->deepface)
(1.9.0)
Requirement already satisfied: click>=8.1.3 in
/usr/local/lib/python3.12/dist-packages (from Flask>=1.1.2->deepface)
(8.3.1)
Requirement already satisfied: itsdangerous>=2.2.0 in
/usr/local/lib/python3.12/dist-packages (from Flask>=1.1.2->deepface)
(2.2.0)
Requirement already satisfied: jinja2>=3.1.2 in
/usr/local/lib/python3.12/dist-packages (from Flask>=1.1.2->deepface)
(3.1.6)
Requirement already satisfied: markupsafe>=2.1.1 in
/usr/local/lib/python3.12/dist-packages (from Flask>=1.1.2->deepface)
(3.0.3)
Requirement already satisfied: werkzeug>=3.1.0 in
/usr/local/lib/python3.12/dist-packages (from Flask>=1.1.2->deepface)
(3.1.3)
Requirement already satisfied: beautifulsoup4 in
/usr/local/lib/python3.12/dist-packages (from gdown>=3.10.1->deepface)
(4.13.5)
Requirement already satisfied: filelock in
/usr/local/lib/python3.12/dist-packages (from gdown>=3.10.1->deepface)
(3.20.3)
Requirement already satisfied: packaging in
/usr/local/lib/python3.12/dist-packages (from gunicorn>=20.1.0->deepface) (26.0rc2)
Requirement already satisfied: absl-py in
/usr/local/lib/python3.12/dist-packages (from keras>=2.2.0->deepface)
(1.4.0)
Requirement already satisfied: rich in /usr/local/lib/python3.12/dist-
packages (from keras>=2.2.0->deepface) (14.2.0)
Requirement already satisfied: namex in
/usr/local/lib/python3.12/dist-packages (from keras>=2.2.0->deepface)
(0.1.0)
Requirement already satisfied: h5py in /usr/local/lib/python3.12/dist-
packages (from keras>=2.2.0->deepface) (3.15.1)
Requirement already satisfied: optree in
/usr/local/lib/python3.12/dist-packages (from keras>=2.2.0->deepface)

```
(0.17.0)
Requirement already satisfied: ml-dtypes in
/usr/local/lib/python3.12/dist-packages (from keras>=2.2.0->deepface)
(0.5.3)
Collecting lightecc (from lightds>=0.0.3->deepface)
  Downloading lightecc-0.0.4-py3-none-any.whl.metadata (14 kB)
Requirement already satisfied: sympy in
/usr/local/lib/python3.12/dist-packages (from lightds>=0.0.3-
>deepface) (1.13.3)
Requirement already satisfied: joblib>=1.4.2 in
/usr/local/lib/python3.12/dist-packages (from mtcnn>=0.1.0->deepface)
(1.5.3)
Collecting lz4>=4.3.3 (from mtcnn>=0.1.0->deepface)
  Downloading lz4-4.4.5-cp312-cp312-
manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.m
etadata (3.8 kB)
Requirement already satisfied: python-dateutil>=2.8.2 in
/usr/local/lib/python3.12/dist-packages (from pandas>=0.23.4-
>deepface) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in
/usr/local/lib/python3.12/dist-packages (from pandas>=0.23.4-
>deepface) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in
/usr/local/lib/python3.12/dist-packages (from pandas>=0.23.4-
>deepface) (2025.2)
Requirement already satisfied: charset_normalizer<4,>=2 in
/usr/local/lib/python3.12/dist-packages (from requests>=2.27.1-
>deepface) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in
/usr/local/lib/python3.12/dist-packages (from requests>=2.27.1-
>deepface) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in
/usr/local/lib/python3.12/dist-packages (from requests>=2.27.1-
>deepface) (2.6.3)
Requirement already satisfied: certifi>=2017.4.17 in
/usr/local/lib/python3.12/dist-packages (from requests>=2.27.1-
>deepface) (2026.1.4)
Requirement already satisfied: astunparse>=1.6.0 in
/usr/local/lib/python3.12/dist-packages (from tensorflow>=1.9.0-
>deepface) (1.6.3)
Requirement already satisfied: flatbuffers>=24.3.25 in
/usr/local/lib/python3.12/dist-packages (from tensorflow>=1.9.0-
>deepface) (25.9.23)
Requirement already satisfied: gast!=0.5.0,!0.5.1,!0.5.2,>=0.2.1
in /usr/local/lib/python3.12/dist-packages (from tensorflow>=1.9.0-
>deepface) (0.6.0)
Requirement already satisfied: google-pasta>=0.1.1 in
/usr/local/lib/python3.12/dist-packages (from tensorflow>=1.9.0-
>deepface) (0.2.0)
```


Requirement already satisfied: libclang>=13.0.0 in
/usr/local/lib/python3.12/dist-packages (from tensorflow>=1.9.0->deepface) (18.1.1)

Requirement already satisfied: opt-einsum>=2.3.2 in
/usr/local/lib/python3.12/dist-packages (from tensorflow>=1.9.0->deepface) (3.4.0)

Requirement already satisfied: protobuf!=4.21.0,!4.21.1,!4.21.2,!4.21.3,!4.21.4,!4.21.5,<6.0.0dev,>=3.20.3 in
/usr/local/lib/python3.12/dist-packages (from tensorflow>=1.9.0->deepface) (5.29.5)

Requirement already satisfied: setuptools in
/usr/local/lib/python3.12/dist-packages (from tensorflow>=1.9.0->deepface) (75.2.0)

Requirement already satisfied: six>=1.12.0 in
/usr/local/lib/python3.12/dist-packages (from tensorflow>=1.9.0->deepface) (1.17.0)

Requirement already satisfied: typing-extensions>=3.6.6 in
/usr/local/lib/python3.12/dist-packages (from tensorflow>=1.9.0->deepface) (4.15.0)

Requirement already satisfied: wrapt>=1.11.0 in
/usr/local/lib/python3.12/dist-packages (from tensorflow>=1.9.0->deepface) (2.0.0)

Requirement already satisfied: grpcio<2.0,>=1.24.3 in
/usr/local/lib/python3.12/dist-packages (from tensorflow>=1.9.0->deepface) (1.75.1)

Requirement already satisfied: tensorboard~=2.19.0 in
/usr/local/lib/python3.12/dist-packages (from tensorflow>=1.9.0->deepface) (2.19.0)

Requirement already satisfied: wheel<1.0,>=0.23.0 in
/usr/local/lib/python3.12/dist-packages (from astunparse>=1.6.0->tensorflow>=1.9.0->deepface) (0.45.1)

Requirement already satisfied: mpmath<1.4,>=1.1.0 in
/usr/local/lib/python3.12/dist-packages (from sympy->lightds>=0.0.3->deepface) (1.3.0)

Requirement already satisfied: markdown>=2.6.8 in
/usr/local/lib/python3.12/dist-packages (from tensorboard~=2.19.0->tensorflow>=1.9.0->deepface) (3.9)

Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in
/usr/local/lib/python3.12/dist-packages (from tensorboard~=2.19.0->tensorflow>=1.9.0->deepface) (0.7.2)

Requirement already satisfied: soupsieve>1.2 in
/usr/local/lib/python3.12/dist-packages (from beautifulsoup4->gdown>=3.10.1->deepface) (2.8)

Requirement already satisfied: PySocks!=1.5.7,>=1.5.6 in
/usr/local/lib/python3.12/dist-packages (from requests[socks]->gdown>=3.10.1->deepface) (1.7.1)

Requirement already satisfied: markdown-it-py>=2.2.0 in
/usr/local/lib/python3.12/dist-packages (from rich->keras>=2.2.0->deepface) (4.0.0)

```

Requirement already satisfied: pygments<3.0.0,>=2.13.0 in
/usr/local/lib/python3.12/dist-packages (from rich->keras>=2.2.0-
>deepface) (2.19.2)
Requirement already satisfied: mdurl~=0.1 in
/usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0-
>rich->keras>=2.2.0->deepface) (0.1.2)
Downloading deepface-0.0.98-py3-none-any.whl (169 kB)
----- 169.2/169.2 kB 5.8 MB/s eta
0:00:00
----- 115.9/115.9 kB 8.6 MB/s eta
0:00:00
----- 169.7/169.7 kB 12.5 MB/s eta
0:00:00
----- 59.4/59.4 kB 4.5 MB/s eta
0:00:00
tcnn-1.0.0-py3-none-any.whl (1.9 MB)
----- 1.9/1.9 MB 38.8 MB/s eta
0:00:0000:01
anylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl
(1.4 MB)
----- 1.4/1.4 MB 62.2 MB/s eta
0:00:00
----- 45.3/45.3 kB 3.2 MB/s eta
0:00:00
tcnn, lightphe, lightdsa, flask-cors, retina-face, deepface
Successfully installed deepface-0.0.98 fire-0.7.1 flask-cors-6.0.2
gunicorn-25.0.0 lightdsa-0.0.3 lightecc-0.0.4 lightphe-0.0.20 lz4-
4.4.5 mtcnn-1.0.0 retina-face-0.0.17
Note: you may need to restart the kernel to use updated packages.

```

```
# @title
```

```

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from deepface import DeepFace
from sklearn.metrics.pairwise import cosine_distances
from tqdm.auto import tqdm
from PIL import Image

```

```

def clean_and_visualize_dataset(df, model_name='Facenet512',
threshold=0.6, max_viz=10):
    """

```

```
    Unified pipeline:
```

1. Detects outliers using facial embeddings and centroid distance.
 2. Visualizes samples of removed images for manual verification.
 3. Prints summary statistics.
 4. Returns the cleaned DataFrame.
- ```
 """
```

```
 indices_to_drop = []
```

```

unique_labels = df["label"].unique()

print(f"--- Initializing Cleaning Pipeline with {model_name} ---")
Warm up the model
_ = DeepFace.build_model(model_name)

1. OUTLIER DETECTION LOOP
for label in tqdm(unique_labels, desc="Scanning Identities",
position=0):
 subset = df[df["label"] == label]

 if len(subset) < 2:
 continue

 img_paths = subset["image_path"].tolist()
 subset_indices = subset.index.tolist()

 embeddings = []
 valid_indices = []

 for i, img_path in enumerate(img_paths):
 try:
 # Optimized for pre-cropped face images
 embedding_obj = DeepFace.represent(
 img_path=img_path,
 model_name=model_name,
 enforce_detection=False,
 detector_backend='skip'
)
 embeddings.append(embedding_obj[0]["embedding"])
 valid_indices.append(subset_indices[i])
 except Exception:
 indices_to_drop.append(subset_indices[i])

 if not embeddings:
 continue

 # Vectorized distance calculation
 embeddings_matrix = np.array(embeddings)
 centroid = np.mean(embeddings_matrix, axis=0).reshape(1, -1)
 dists = cosine_distances(embeddings_matrix,
centroid).flatten()

 # Identify local outliers based on threshold
 outlier_locs = np.where(dists > threshold)[0]
 for loc in outlier_locs:
 indices_to_drop.append(valid_indices[loc])

2. VISUALIZATION OF OUTLIERS
if indices_to_drop:

```

```

 print(f"\n--- Visualizing up to {max_viz} Outlier Samples
 ---")
 viz_indices = indices_to_drop[:max_viz]
 plt.figure(figsize=(15, 5))
 for i, idx in enumerate(viz_indices):
 row = df.loc[idx]
 img = Image.open(row["image_path"]).convert("RGB")
 plt.subplot(1, max_viz, i + 1)
 plt.imshow(img)
 plt.title(f"ID: {row['label']}\nOutlier", fontsize=8)
 plt.axis("off")
 plt.tight_layout()
 plt.show()
else:
 print("\n No outliers found with current threshold.")

3. CLEANING & SUMMARY
df_clean = df.drop(indices_to_drop).reset_index(drop=True)

n_original = len(df)
n_removed = len(indices_to_drop)
perc_removed = (n_removed / n_original) * 100

classes_before = df["label"].nunique()
classes_after = df_clean["label"].nunique()

print(f"\n" + "="*30)
print(f"CLEANING SUMMARY")
print(f"="*30)
print(f"Original Images: {n_original}")
print(f"Removed Images: {n_removed} ({perc_removed:.2f}%)")
print(f"Final Image Count: {len(df_clean)}")
print(f"Identities Lost: {classes_before - classes_after}")
print(f"="*30 + "\n")

return df_clean

```

2026-02-01 14:28:57.834508: E

external/local\_xla/xla/stream\_executor/cuda/cuda\_fft.cc:467] Unable to register cuFFT factory: Attempting to register factory for plugin cuFFT when one has already been registered

WARNING: All log messages before absl::InitializeLog() is called are written to STDERR

E0000 00:00:1769956138.041576 55 cuda\_dnn.cc:8579] Unable to register cuDNN factory: Attempting to register factory for plugin cuDNN when one has already been registered

E0000 00:00:1769956138.101747 55 cuda\_blas.cc:1407] Unable to register cuBLAS factory: Attempting to register factory for plugin cuBLAS when one has already been registered

W0000 00:00:1769956138.591935 55 computation\_placer.cc:177]

```
computation placer already registered. Please check linkage and avoid
linking the same target more than once.
W0000 00:00:1769956138.591975 55 computation_placer.cc:177]
computation placer already registered. Please check linkage and avoid
linking the same target more than once.
W0000 00:00:1769956138.591978 55 computation_placer.cc:177]
computation placer already registered. Please check linkage and avoid
linking the same target more than once.
W0000 00:00:1769956138.591980 55 computation_placer.cc:177]
computation placer already registered. Please check linkage and avoid
linking the same target more than once.

26-02-01 14:29:11 - Directory /root/.deepface has been created
26-02-01 14:29:11 - Directory /root/.deepface/weights has been created
```

## Data

For the experiments, I decided to use several datasets available on Kaggle, prioritizing datasets that were already processed in terms of:

- **Alignment:** face patterns were approximately in consistent positions.
  - **Cropping:** images contained only the face, avoiding background or body parts.
- 

## Main training dataset

**VGGFace2:**

<https://www.kaggle.com/datasets/yakhyokhuja/vggface2-112x112>

This dataset is a **standard for training face recognition models**. Key characteristics:

- 8631 identities
- Average of 362.6 images per identity

Due to **computational limitations**, the base setup I used is:

- 3000 identities
  - 80 images per identity
- 

## Validation dataset

**PINS Face Recognition** dataset:

<https://www.kaggle.com/datasets/hereisburak/pins-face-recognition>

Key characteristics:

- 105 identities of famous people collected by Pinterest
- Number of images per identity varies between 80 and 240

This dataset provides a **good base to monitor the model's learning progress**.

---

## Additional datasets for experiments

To test the model's learning and compare performance on different data distributions, I also used:

1. The same validation dataset (**PINS Face Recognition**).
2. **Face Recognition Dataset - Oneshot Learning:**  
<https://www.kaggle.com/datasets/stoicstatic/face-recognition-dataset>Another:
  - Dataset with 1324 identities
  - About half of the identities have **only 2 images**
  - Almost all identities have **no more than 10 images**
3. **Face Recognition Dataset:**  
<https://www.kaggle.com/datasets/vasukipatel/face-recognition-dataset>
  - 31 identities
  - Number of images per identity between 70 and 100

This setup allows us to:

- Compare model performance on **different datasets**
- Analyze how **data distribution** (number of images per identity) affects evaluation metrics

```
@title
import numpy as np
import pandas as pd
import os
from tqdm.auto import tqdm

ROOT_DIR = "/kaggle/input/face-recognition-dataset/Extracted
Faces/Extracted Faces"

image_paths = []
labels = []

Prendiamo SOLO le cartelle id_XXX
id_folders = sorted([
 d for d in os.listdir(ROOT_DIR)
```

```

 if os.path.isdir(os.path.join(ROOT_DIR, d))
])

Barra 1: progresso sulle cartelle
for id_folder in tqdm(id_folders, desc="Cartelle (identità)",
position=0, mininterval=1.0):

 folder_path = os.path.join(ROOT_DIR, id_folder)

 images = [
 f for f in os.listdir(folder_path)
 if f.lower().endswith((".jpg", ".png", ".jpeg"))
]

 # Barra 2: immagini nella cartella corrente
 for img_name in tqdm(
 images,
 desc=f"Immagini in {id_folder}",
 position=1,
 leave=False
):
 full_path = os.path.join(folder_path, img_name)

 image_paths.append(full_path)
 labels.append(id_folder)

Costruzione DataFrame
df_test_1= pd.DataFrame({
 "image_path": image_paths,
 "label": labels
})

print(df_test_1.head())
print("Numero totale immagini:", len(df_test_1))
print("Numero classi:", df_test_1["label"].nunique())

df_test_1 = clean_and_visualize_dataset(df_test_1, threshold=0.6)

@title
import os
import pandas as pd
from tqdm import tqdm

Path al dataset
DATASET_DIR =
"/kaggle/input/d/vasukipatel/face-recognition-dataset/Faces/Faces"

image_paths = []
labels = []

```

```

Lista file (tqdm mostra il progresso)
for filename in tqdm(os.listdir(DATASET_DIR), desc="Scansione immagini"):
 if filename.lower().endswith((".jpg", ".png", ".jpeg")):
 # path completo immagine
 image_path = os.path.join(DATASET_DIR, filename)

 # label = nome personaggio (prima di "_")
 label = filename.rsplit("_", 1)[0]

 image_paths.append(image_path)
 labels.append(label)

df_test_2 = pd.DataFrame({
 "image_path": image_paths,
 "label": labels
})

print(df_test_2.head())
print("Numero immagini:", len(df_test_2))
print("Numero classi:", df_test_2["label"].nunique())

df_test_2 = clean_and_visualize_dataset(df_test_2, threshold=0.6)

@title
ROOT_DIR =
"/kaggle/input/pins-face-recognition/105_classes_pins_dataset"

image_paths = []
labels = []

Prendiamo SOLO le cartelle id_xxx
id_folders = sorted([
 d for d in os.listdir(ROOT_DIR)
 if os.path.isdir(os.path.join(ROOT_DIR, d))
])

▢ Barra 1: progresso sulle cartelle
for id_folder in tqdm(id_folders, desc="Cartelle (identità)",
position=0, mininterval=1.0):

 folder_path = os.path.join(ROOT_DIR, id_folder)

 images = [
 f for f in os.listdir(folder_path)
 if f.lower().endswith((".jpg", ".png", ".jpeg"))
]

 # ▢ Barra 2: immagini nella cartella corrente
 for img_name in tqdm(

```



```

 images,
 desc=f"Immagini in {id_folder}",
 position=1,
 leave=False
):
 full_path = os.path.join(folder_path, img_name)

 image_paths.append(full_path)
 labels.append(id_folder)

Costruzione DataFrame
df_val= pd.DataFrame({
 "image_path": image_paths,
 "label": labels
})

print(df_val.head())
print("Numero totale immagini:", len(df_val))
print("Numero classi:", df_val["label"].nunique())

#df_val = clean_and_visualize_dataset(df_val, threshold=0.6)

@title Data inspection
all_labels = pd.concat([
 df_train_small['label'],
 df_val['label'],
 df_test_1['label'],
 df_test_2['label']
])

label_to_idx = {label: idx for idx, label in
enumerate(all_labels.unique())}

df_train_small['label_idx'] =
df_train_small['label'].map(label_to_idx)
df_val['label_idx'] = df_val['label'].map(label_to_idx)
df_test_1['label_idx'] = df_test_1['label'].map(label_to_idx)
df_test_2['label_idx'] = df_test_2['label'].map(label_to_idx)

num_classes_total = len(label_to_idx)
print(f"Total number of classes: {num_classes_total}")

=====
DATASET STATISTICS
=====
def dataset_stats(df, name="DATASET"):
 """
 Prints descriptive statistics about the label distribution in a

```

```

dataframe
and displays a histogram.
"""
counts = df['label'].value_counts()

print(f"\n===== STATISTICS: {name} =====")
print(f"Total images: {len(df)}")
print(f"Total classes: {counts.shape[0]}")
print(f"Min images per class: {counts.min()}")
print(f"Max images per class: {counts.max()}")
print(f"Mean images per class: {counts.mean():.2f}")
print(f"Median images per class: {counts.median()}")
print(f"25th percentile: {counts.quantile(0.25)}")
print(f"75th percentile: {counts.quantile(0.75)}")

plt.figure(figsize=(10, 4))
plt.hist(counts, bins=50, color='skyblue', edgecolor='black')
plt.title(f"Image Distribution per Class ({name})")
plt.xlabel("Number of images")
plt.ylabel("Number of classes")
plt.grid(axis='y', alpha=0.3)
plt.show()

dataset_stats(df_train_small, name="TRAIN")
dataset_stats(df_val, name="VAL")
dataset_stats(df_test_1, name="TEST 1")
dataset_stats(df_test_2, name="TEST 2")

class FaceDataset(torch.utils.data.Dataset):
 def __init__(self, df, transform=None):
 self.df = df.reset_index(drop=True)
 self.transform = transform

 def __len__(self):
 return len(self.df)

 def __getitem__(self, idx):
 row = self.df.iloc[idx]
 img_path = row['image_path']
 label = row['label_idx']

 # Load image and ensure it is in RGB format
 img = Image.open(img_path).convert("RGB")

 if self.transform:
 img = self.transform(img)

 return img, label

```

```

=====
DATA TRANSFORMS (AUGMENTATION)
=====

train_tfms = transforms.Compose([
 transforms.Resize((112, 112)),
 transforms.ColorJitter(brightness=0.1, contrast=0.1,
saturation=0.1),
 transforms.RandomRotation(degrees=5),
 transforms.ToTensor(),
 transforms.Normalize(mean=[0.5, 0.5, 0.5],
 std=[0.5, 0.5, 0.5]),
 transforms.RandomErasing(p=0.1, scale=(0.02, 0.1))
])

val_test_tfms = transforms.Compose([
 transforms.Resize((112, 112)),
 transforms.ToTensor(),
 transforms.Normalize(mean=[0.5, 0.5, 0.5],
 std=[0.5, 0.5, 0.5])
])

=====
DATASET INSTANTIATION
=====
train_dataset = FaceDataset(df_train_small, transform=train_tfms)
val_dataset = FaceDataset(df_val, transform=val_test_tfms)
test_dataset_1 = FaceDataset(df_test_1, transform=val_test_tfms)
test_dataset_2 = FaceDataset(df_test_2, transform=val_test_tfms)

=====
DATALOADERS
=====
BATCH_SIZE = 256
NUM_WORKERS = 4

train_loader = DataLoader(
 train_dataset,
 batch_size=BATCH_SIZE,
 shuffle=True,
 num_workers=NUM_WORKERS,
 pin_memory=True,
 drop_last=True
)

val_loader = DataLoader(
 val_dataset,
 batch_size=BATCH_SIZE,
 shuffle=False,

```

```

 num_workers=NUM_WORKERS
)

 test_loader_1 = DataLoader(
 test_dataset_1,
 batch_size=BATCH_SIZE,
 shuffle=False,
 num_workers=NUM_WORKERS
)

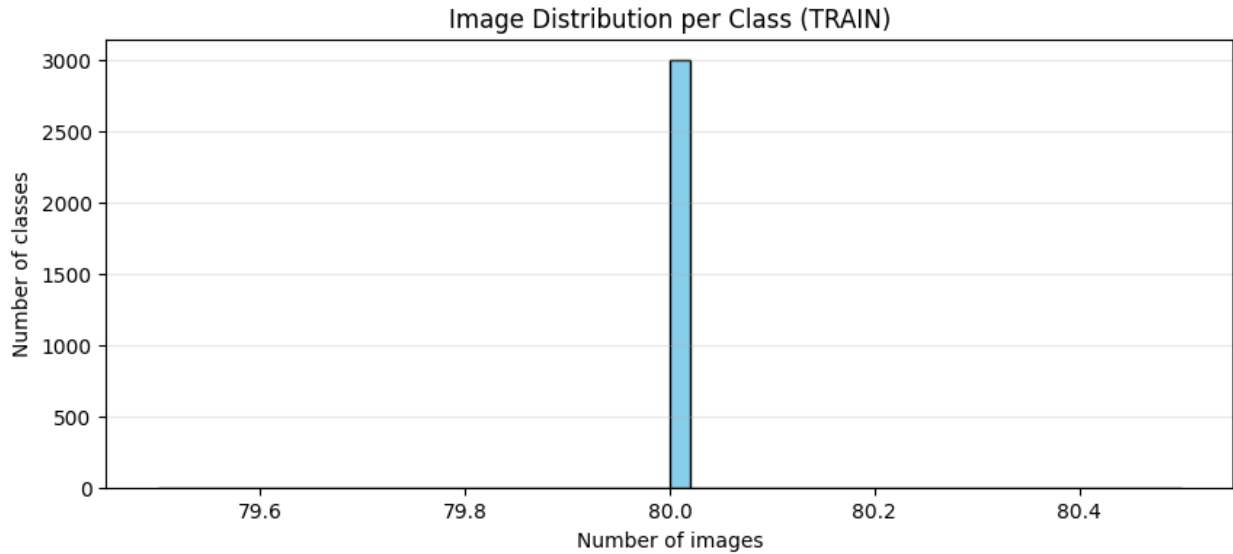
 test_loader_2 = DataLoader(
 test_dataset_2,
 batch_size=BATCH_SIZE,
 shuffle=False,
 num_workers=NUM_WORKERS
)

 print("\nDataLoaders are ready.")
 print(f"Images in TRAIN: {len(train_dataset)}")
 print(f"Images in VAL: {len(val_dataset)}")
 print(f"Images in TEST 1: {len(test_dataset_1)}")
 print(f"Images in TEST 2: {len(test_dataset_2)}")

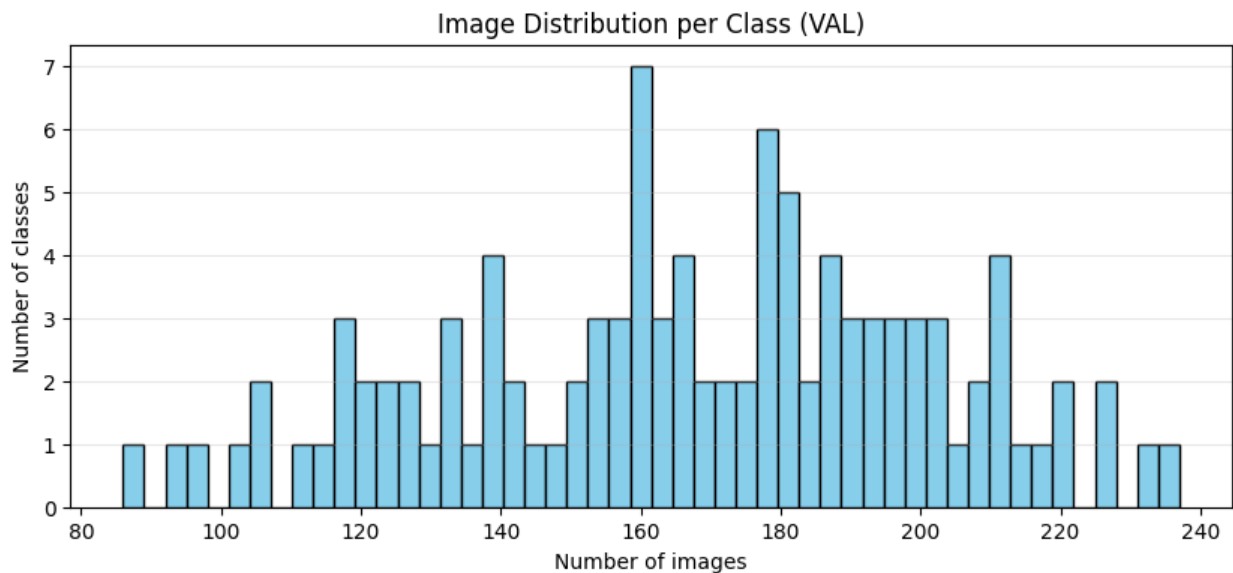
 Total number of classes: 4460

 ===== STATISTICS: TRAIN =====
 Total images: 240000
 Total classes: 3000
 Min images per class: 80
 Max images per class: 80
 Mean images per class: 80.00
 Median images per class: 80.0
 25th percentile: 80.0
 75th percentile: 80.0

```

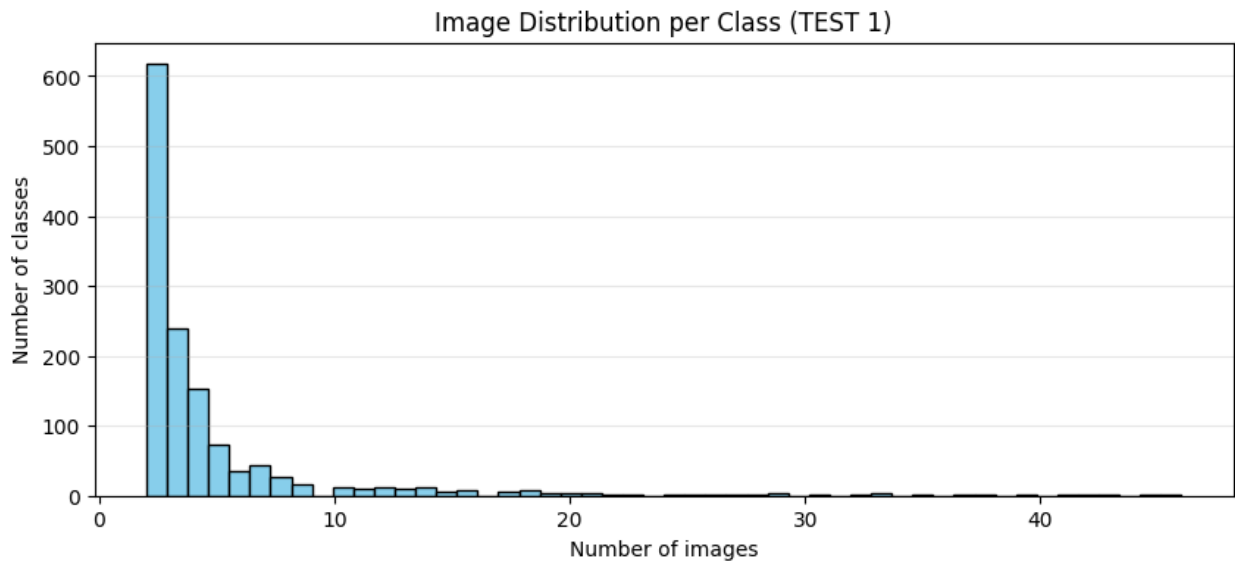


```
===== STATISTICS: VAL =====
Total images: 17534
Total classes: 105
Min images per class: 86
Max images per class: 237
Mean images per class: 166.99
Median images per class: 169.0
25th percentile: 141.0
75th percentile: 192.0
```



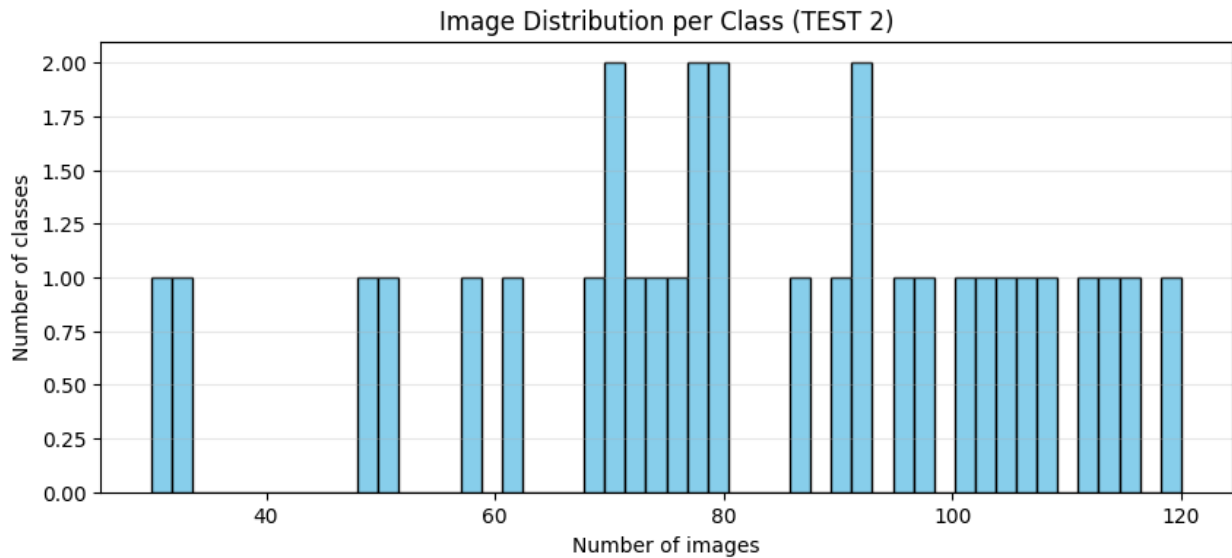
```
===== STATISTICS: TEST 1 =====
Total images: 6107
```

Total classes: 1324  
Min images per class: 2  
Max images per class: 46  
Mean images per class: 4.61  
Median images per class: 3.0  
25th percentile: 2.0  
75th percentile: 4.0



==== STATISTICS: TEST 2 ====

Total images: 2562  
Total classes: 31  
Min images per class: 30  
Max images per class: 120  
Mean images per class: 82.65  
Median images per class: 80.0  
25th percentile: 70.5  
75th percentile: 101.5



```
DataLoaders are ready.
Images in TRAIN: 240000
Images in VAL: 17534
Images in TEST 1: 6107
Images in TEST 2: 2562
```

```
@title Visualization
import matplotlib.pyplot as plt
from PIL import Image

def visualize_dataset_samples(df, dataset_name="Dataset"):
 """
 Visualizes two things from the provided dataframe:
 1. A grid of 10 random images from different identities.
 2. A grid showing 5 images each for 3 specific random identities.
 """
 print(f"\n--- Visualizing: {dataset_name} ---")

 # --- PART 1: Plot 10 random identities ---
 # We sample 10 unique rows
 sample_random = df.sample(10)

 plt.figure(figsize=(15, 6))
 plt.suptitle(f"{dataset_name}: 10 Random Samples", fontsize=16)

 for i, (_, row) in enumerate(sample_random.iterrows()):
 img = Image.open(row["image_path"]).convert("RGB")
 plt.subplot(2, 5, i + 1)
 plt.imshow(img)
 plt.title(f"ID: {row['label']}", fontsize=8)
 plt.axis("off")
```

```

plt.tight_layout()
plt.show()

--- PART 2: Plot 5 images for 3 specific identities ---
Pick 3 random unique labels/identities from the dataset
unique_labels = df["label"].unique()
selected_identities = np.random.choice(unique_labels, size=3,
replace=False)

plt.figure(figsize=(15, 9))
plt.suptitle(f"{dataset_name}: Consistency Check (3 Identities, 5
Images each)", fontsize=16)

plot_idx = 1
for label in selected_identities:
 # Filter all images belonging to this identity
 subset = df[df["label"] == label]

 # Sample 5 images (with replacement=True only if the ID has
fewer than 5 images)
 replace_mode = len(subset) < 5
 identity_samples = subset.sample(5, replace=replace_mode)

 for _, row in identity_samples.iterrows():
 img = Image.open(row["image_path"]).convert("RGB")
 plt.subplot(3, 5, plot_idx)
 plt.imshow(img)
 plt.title(f"Label: {label}", fontsize=9)
 plt.axis("off")
 plot_idx += 1

plt.tight_layout()
plt.show()

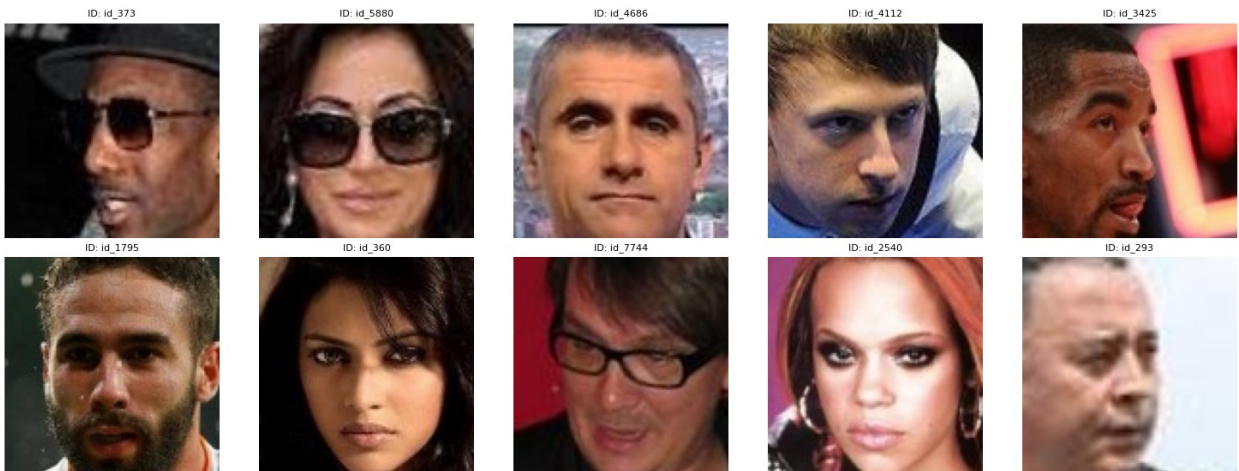
=====
EXECUTION
=====
You can now call this function for each of your dataframes:
visualize_dataset_samples(df_train_small, "TRAIN SET")
visualize_dataset_samples(df_val, "VAL SET 1")
visualize_dataset_samples(df_test_1, "TEST SET 1")
visualize_dataset_samples(df_test_2, "TEST SET 2")

--- Visualizing: TRAIN SET ---

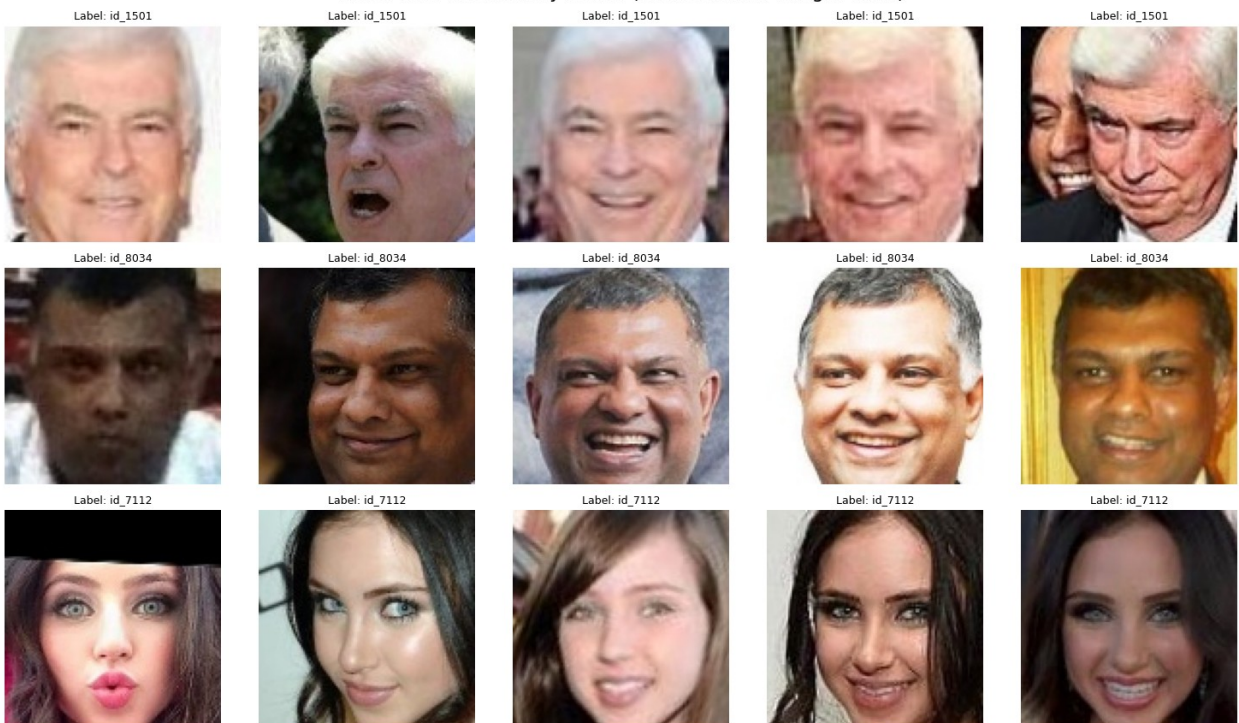
```



TRAIN SET: 10 Random Samples

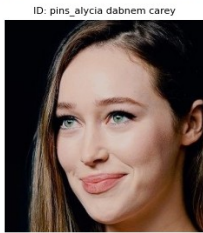
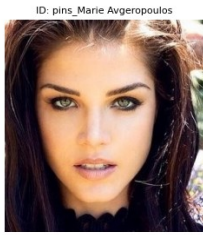


TRAIN SET: Consistency Check (3 Identities, 5 Images each)

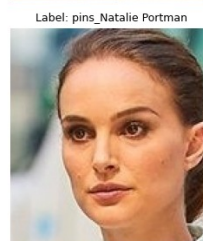
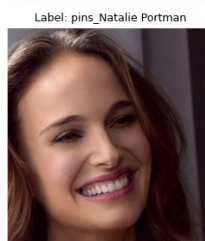
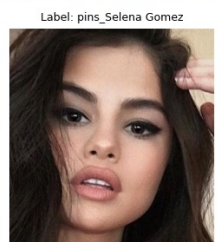
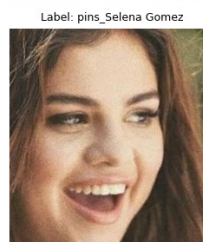
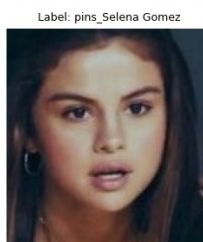
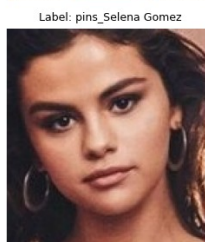
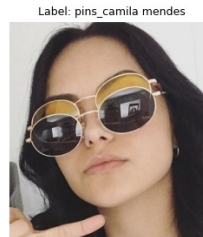
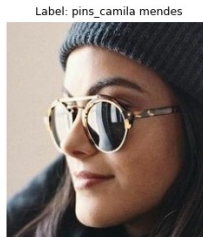
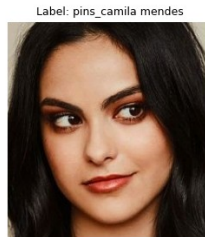
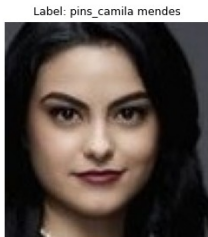


--- Visualizing: VAL SET 1 ---

### VAL SET 1: 10 Random Samples



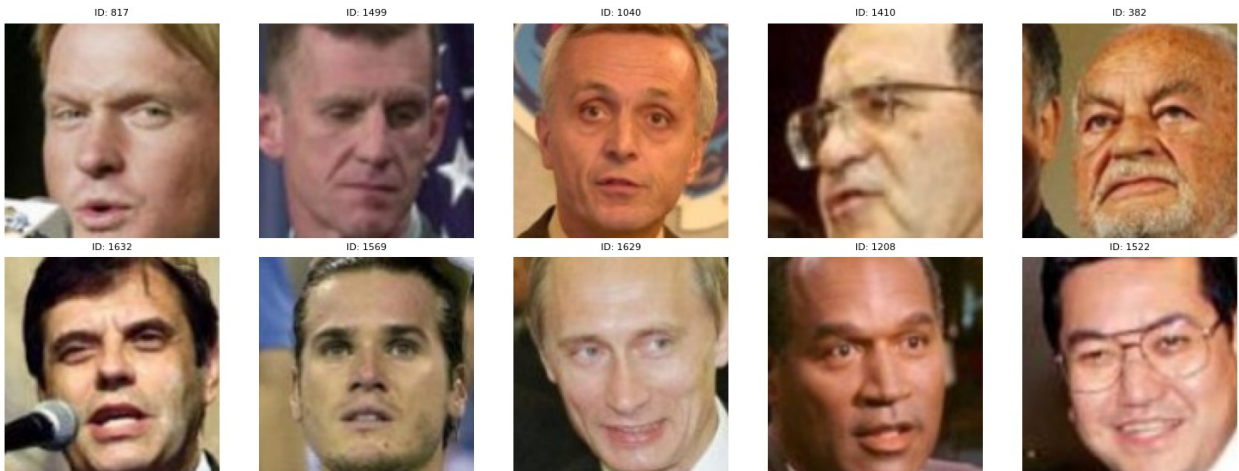
### VAL SET 1: Consistency Check (3 Identities, 5 Images each)



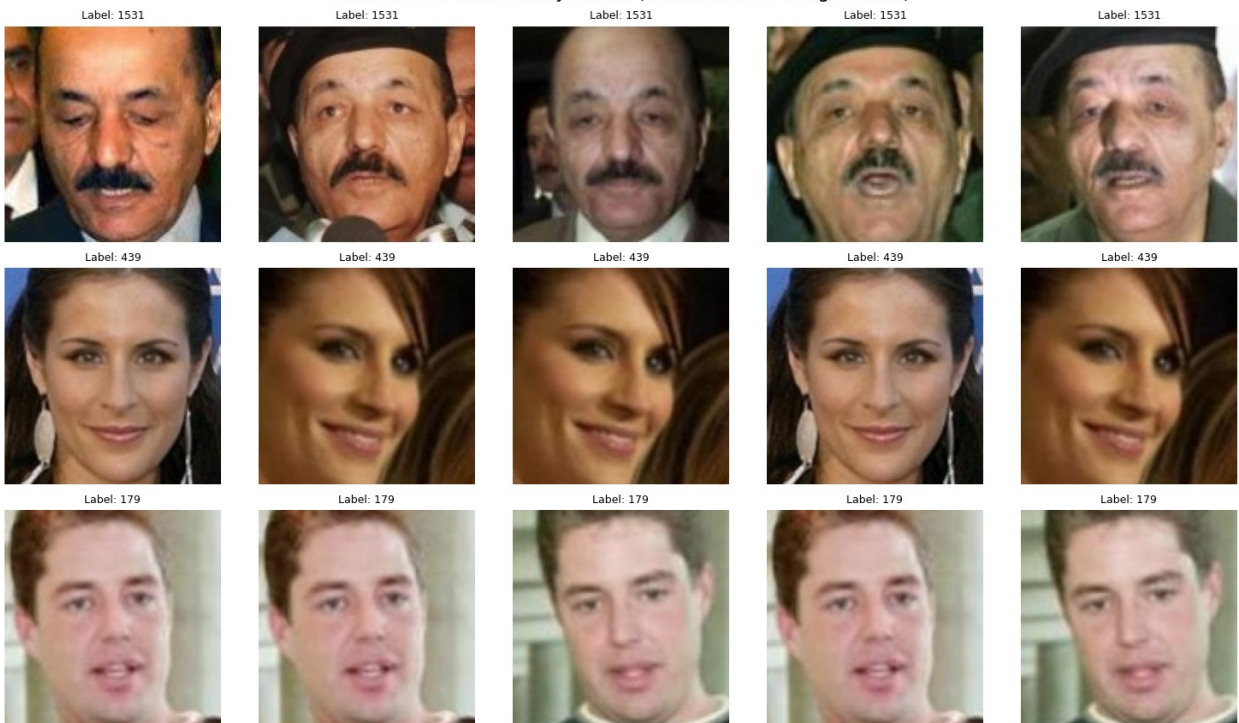
--- Visualizing: TEST SET 1 ---



TEST SET 1: 10 Random Samples

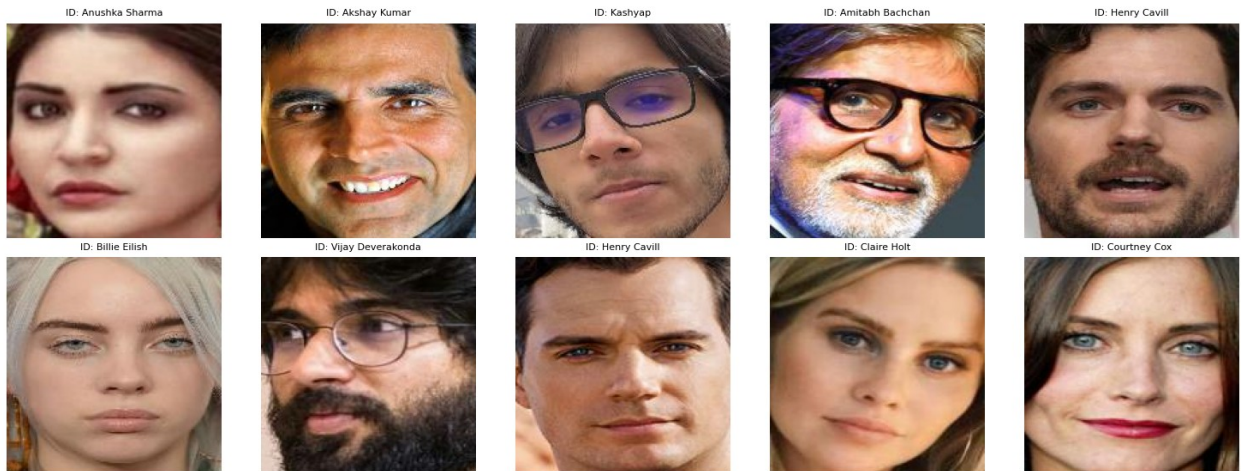


TEST SET 1: Consistency Check (3 Identities, 5 Images each)



--- Visualizing: TEST SET 2 ---

TEST SET 2: 10 Random Samples



TEST SET 2: Consistency Check (3 Identities, 5 Images each)



## Training Setup

For training, the **reduced VGGFace2 dataset** was used as discussed before. A **limited data augmentation** strategy was applied to make the model slightly more robust, without significantly complicating the training process.

The choices of **batch size** and **optimizer** were primarily aimed at achieving **convergence in a reasonable time**, as the goal was not to obtain the most performant models, but rather to observe the behavior of the loss functions. Specifically:

- **Batch size:** 256 (instead of the standard 512).
  - **Optimizer:** AdamW was used to accelerate convergence, while in the literature **SGD with momentum** is more commonly used; SGD can reach slightly better optima but requires much slower convergence.
- 

## Experiments

During training, different configurations were explored:

- **Loss functions:** Adacos, ArcFace and CosFace with different **scale factor values**, keeping the same margin.
  - **Dataset size impact:** AdaCos was also trained using **6000 identities with 150 images per identity** to investigate how the amount of training data affects performance.
- 

## Validation Metric: TAR@FAR

To monitor the model during validation, the **TAR@FAR** metric was used with **FAR = 1e-3**. The metric was computed as follows:

1. **Embedding Extraction:** All validation images are passed through the model to obtain L2-normalized embeddings.
2. **Similarity Calculation:** A **cosine similarity matrix** is computed between all embeddings.
3. **Pair Separation:**
  - Positive pairs (same identity) are identified from the labels.
  - Negative pairs (different identities) are identified and sampled (because the number of negative pairs was extremely large, just 1 million negatives were sampled for efficiency).
4. **Threshold Selection:** The similarity threshold that achieves the target **FAR = 1e-3** is computed (0.1% of error is allowed, in that case 1000 errors are allowed over 1M negatives).
5. **TAR Computation:** The **True Acceptance Rate (TAR)** is then calculated as the fraction of positive pairs whose similarity exceeds this threshold.

The full **training code** used to run these experiments is provided at the end of the notebook.



# Model

The model I chose is **SE-ResNet50 (seresnet50)** from the [timm](#) library as the backbone, a standard architecture adopted in the literature for this type of task.

The architecture is composed of two main components:

- **Backbone:** responsible for extracting spatial features from the faces.
- **Global Average Pooling** to transform the feature map form (C, H, W) to (C, 1, 1)
- **Projection Head:** transforms the extracted features into the embedding of a choosen size.

```
class FaceModel(nn.Module):
 def __init__(self, model_name='seresnet50', embedding_size=512,
pretrained=True, use_gdc=False):
 super().__init__()

 self.backbone = timm.create_model(model_name,
pretrained=pretrained, num_classes=0, global_pool='')

 # feature dimensions computation
 self.backbone.eval()
 with torch.no_grad():
 dummy = torch.zeros(1, 3, 112, 112)
 feat_map = self.backbone(dummy)
 self.in_channels = feat_map.shape[1]
 self.feats_h = feat_map.shape[2]
 self.feats_w = feat_map.shape[3]
 self.backbone.train()

 # Pooling Strategy
 self.use_gdc = use_gdc
 if self.use_gdc:
 # GDC: Global Depthwise Convolution (Not used)
 self.gdc = nn.Sequential(
 nn.Conv2d(self.in_channels, self.in_channels,
kernel_size=(self.feats_h, self.feats_w),
 groups=self.in_channels, bias=False),
 nn.BatchNorm2d(self.in_channels),
 nn.PReLU(self.in_channels)
)
 else:
 self.gap = nn.AdaptiveAvgPool2d(1)

 # projection head (BN-Neck)
 self.head = nn.Sequential(
 nn.Flatten(),
 #nn.Dropout(p=0.4),
 nn.Linear(self.in_channels, embedding_size, bias=False),
```

```

 nn.BatchNorm1d(embedding_size)
)
 self._init_weights()

 def _init_weights(self):
 for m in self.head.modules():
 if isinstance(m, nn.Linear):
 nn.init.xavier_uniform_(m.weight)
 elif isinstance(m, nn.BatchNorm1d) or isinstance(m,
nn.BatchNorm2d):
 nn.init.constant_(m.weight, 1)
 nn.init.constant_(m.bias, 0)

 def forward(self, x):
 x = self.backbone(x)
 if self.use_gdc:
 x = self.gdc(x)
 else:
 x = self.gap(x)

 x = self.head(x)

 return x

```

## Note about the training experiments

# Analysis of Training Results with Different Scale Factors

---

## Brief recap of the scale factor effect

In general, the effect of the scale factor can be described as follows:

### Effect of a **Large Scale Factor**

1. Strong gradients when the model makes wrong predictions: at the beginning of training, embeddings are poorly organized, and it is common for the model to assign a **higher cosine similarity to a wrong class than to the correct one**.

*Example:*

- Correct class cosine similarity: 0.15
- Incorrect class cosine similarity: 0.20

Large scale factor ( $s=50$ )

$$\text{softmax}(s \cdot [0.15, 0.20]) = \text{softmax}([7.5, 10.0]) \approx [0.076, 0.924]$$

The model is very confident in the **wrong class**, and the loss produces large gradients. This strong gradient forces the model to rapidly separate the embeddings, aggressively correcting early mistakes.

1. Early saturation of probabilities even with low cosine similarity

A drawback of a large scale factor is that it can saturate the softmax probabilities too early, even when cosine similarities are still relatively low and far from optimal alignment.

*Example:*

- Correct class cosine similarity: 0.25
- Next highest cosine similarity: 0.20

Large scale factor ( $s=50$ )

$$\text{softmax}(s \cdot [0.25, 0.20]) = \text{softmax}([12.5, 10.0]) \approx [0.924, 0.076]$$

Despite the correct cosine similarity being only 0.25, the predicted probability is already close to 1.

As a consequence, the **gradient becomes very small**, and the model receives little incentive to further improve the angular alignment.

## Effect of a **Small Scale Factor**

1. Weak gradients when the model makes wrong predictions: at the beginning of training, when embeddings are still poorly separated and wrong predictions are common, a small scale factor generates weaker gradients compared to a large one.

**Example:**

- Correct class cosine similarity: 0.15
- Incorrect class cosine similarity: 0.20

Small scale factor ( $s=10$ )

$$\text{softmax}(s \cdot [0.15, 0.20]) = \text{softmax}([1.5, 2.0]) \approx [0.377, 0.623]$$

The model assigns a higher probability to the wrong class, but the difference between probabilities is lower than the case seen before.

As a result, the gradient signal is relatively weak.

1. Sustained learning signal when predictions are already good

A key advantage of a small scale factor emerges when the model starts producing **qualitatively correct predictions**.

**Example:**



- Correct class cosine similarity: 0.25
- Next highest cosine similarity: 0.20

Small scale factor ( $s=10$ )

$$\text{softmax}(s \cdot [0.25, 0.20]) = \text{softmax}([2.5, 2.0]) \approx [0.622, 0.378]$$

Even though the prediction is correct, the probability assigned to the target class is far from saturation.

This results in **non-negligible gradients**, which continue to push the model to improve the angular alignment between embeddings and class prototypes.

---

In light of the considerations discussed above, we can now analyze the results obtained by the different loss functions when using scale factors of varying magnitude.

During training with AdaCos, the scale factor follows the evolution shown in the figure. Since AdaCos is specifically designed to continuously encourage the model to improve the quality of its representations, the observed range of scale factor values can be regarded as a meaningful estimate of the most suitable scale factor range for the considered training configuration.

The following figure shows the cosine similarity distributions of the validation predictions obtained with the best model before early stopping.

The **green distribution** represents similarity scores for **same-identity (genuine) pairs**, while the **red distribution** corresponds to **different-identity (impostor) pairs**.

This visualization becomes particularly informative when compared with the distributions obtained from the other training configurations discussed later.

From the figure, several relevant observations can be made:

- The **right tail of the non-target (adversarial) distribution** is relatively long and extends to cosine similarity values close to **0.6**.
- More generally, the wide **overlapping region between the two distributions** indicates that, although the model is able to learn useful patterns to correctly recognize some matches with relatively high similarity values, it still shows **difficulties in distinguishing a significant number of identities**.

It is also worth noting that, although in the AdaCos paper the range of scale factor values observed during training (in one of their experiments) lies in a similar interval, in our setting — which is characterized by a more limited training configuration — this behavior appears to translate into a **reduced ability to penalize difficult cases**, where the differences between faces are more subtle and less evident.

This result highlights an important trade-off introduced by the AdaCos training philosophy:

- the use of a **moderate and adaptive scale factor** allows the model to keep refining correct predictions and to avoid early saturation;

- at the same time, this choice **reduces the strength of the gradients on hard negatives**, leading to a **less clear separation between difficult classes**.
- 

For **ArcFace** and **CosFace**, I decided to experiment with three different values of the scale factor:  $s=15$ ,  $s=30$ , and  $s=50$ . The choice of  $s=15$  is motivated by the fact that it represents a meaningful value for comparison with **AdaCos**. The values  $s=30$  and  $s=50$  are instead selected because they are commonly reported as reference values in the original ArcFace and CosFace papers

#### Note on the margin selection

In all the experiments involving **ArcFace**, the margin parameter was fixed to  $m=0.5$ , while for **CosFace** the margin was set to  $m=0.35$ .

These values were selected according to the respective original papers, where they are reported as the most effective margins for the two loss formulations.

### Scale factor $s=15$

Since a scale factor comparable to the one observed in AdaCos is used, the resulting distributions are also qualitatively similar. This behavior will be analyzed in more detail in the following section, where the effect of the **margin** is explicitly discussed.

### Scale factor $s=30/50$

ArcFace

CosFace

From the reported figures, the distributions appear slightly different from those obtained when the scale factor is kept at lower values.

These results suggest that, in this configuration, the use of a large scale factor introduces a discrepancy between the model behavior during training and the performance observed at validation time.

In particular, although the model seems to converge during training — producing predictions whose angular distance from the target class center is typically around **40 degrees** — the resulting angular configurations appear qualitatively worse.

My interpretation is that a large scale factor effectively projects the embeddings onto a **hypersphere that is too large**.

In this setting, class separation during training becomes relatively easy, because the available angular space is wide and the model can satisfy the loss constraints without learning truly discriminative representations.

In practice, when  $s=15$ , the feature space forces the model to learn finer and more detailed information in order to form well-separated clusters. However, this value is not strong enough to properly penalize very similar identities. On the other hand, with a larger value of  $s$ , the model focuses more on separating different identities and can create clusters more efficiently. However, in this case, the model tends to learn coarse patterns and does not capture all the facial details needed to form compact clusters. As a result, the similarity between images of the

same or very similar identities can be low, because the model is mainly optimized to distinguish different identities rather than to model fine intra-class variations.

From the training point of view, this leads to an initial phase in which the model can strongly penalize wrong predictions, producing large gradients that allow the network to quickly learn the first discriminative patterns. However, once the clusters are roughly defined, the model tends to **overfit**, probably learning patterns that are useful to better separate training identities but they can't work at inference time on new unseen data.

Finally, the fact that both the target and non-target similarity distributions remain concentrated at relatively low values indicates that the overall learning process is a bit limited.

As discussed above, the embeddings learned for the target identities still appear to rely on simple and coarse patterns, which are only sufficient to produce a minimal separation between matching and non-matching identities, leaving a large overlap between the two distributions.

## Analysis of the Margin Effect

The analysis of the margin is carried out using a fixed scale factor  $s=15$ , since this value appears to provide more meaningful feature representations.

To understand the role of the margin, I compared a model trained with **no margin** and the results obtained using **ArcFace** and **CosFace**.

By observing the resulting distributions and comparing them with the **no-margin** case, we can note that:

- the behaviour obtained with **CosFace** is very similar to the one observed without margin, reached with a **faster convergence**;
- in contrast, the model trained with **ArcFace** shows a slightly **different distribution shape**.

A possible explanation for this behaviour can be found by analysing the **derivative of the loss with respect to the angle  $\theta$**  of the correct class.

In general, the gradient with respect to  $\theta$  is composed of:

- the **scale factor  $s$** ;
  - a term related to the **classification error**;
  - the **derivative of the logit** with respect to the angle.
- 

## Gradient in CosFace

In CosFace, the logit of the correct class is defined as

$$f(\theta) = \cos \theta - m.$$

The gradient of the loss with respect to the angle is

$$\frac{\partial L}{\partial \theta} = s \cdot (P - 1) \cdot (-\sin \theta).$$

Rewriting it to make the contributions clearer:

$$\frac{\partial L}{\partial \theta} = s \cdot \underset{i}{(1 - P)} \cdot \underset{i}{\sin \theta}.$$

Where:

- $s$  directly scales the magnitude of the gradient;
- $(1 - P)$  represents the classification error, with  $P$  equal to:

$$P = \frac{e^{s(\cos \theta - m)}}{e^{s(\cos \theta - m)} + \sum_k e^{z_k}}$$

- since the margin  $m$  reduces the logit of the correct class, the value of  $P$  tends to remain lower, and therefore the term  $(1 - P)$  stays large for a longer time, keeping the model under stronger training pressure;
- the geometric term  $\sin \theta$  is the same as in a standard softmax formulation (derivative of a constant is equal to zero).

In practice, in CosFace the margin mainly affects the **error term**, while the geometric structure of the gradient remains unchanged.

---

## Gradient in ArcFace

In ArcFace, the logit is defined as

$$f(\theta) = \cos(\theta + m).$$

The gradient becomes

$$\frac{\partial L}{\partial \theta} = s \cdot (P - 1) \cdot (-\sin(\theta + m)).$$

Rewriting it,

$$\frac{\partial L}{\partial \theta} = s \cdot \underset{i}{(1 - P)} \cdot \underset{i}{\sin(\theta + m)}.$$

The main difference lies in the geometric term

$$\sin(\theta + m).$$

In ArcFace, the gradient does not depend only on the current angle  $\theta$ , but on an angle that is **shifted** by a fixed value  $m$ .

---

From the observed distributions, it can be noted that the ArcFace margin allows the distribution of the target predictions to keep a slightly higher mean, which indicates a better average intra-class similarity. At the same time, a slightly longer left tail can be observed in the distribution of **non-target predictions**, which suggests a stronger push to move different identities towards lower similarity values.

The following figure shows the angular margin applied by ArcFace (constant) and by CosFace (variable, depending on the current prediction).

The same experiment was repeated by increasing the margin values to **6.5 for ArcFace** and **5 for CosFace**. The resulting distributions are shown below.

The differences with respect to the standard configuration are very small and are limited to a slight change in the shape of the distributions, without improvements.

This result suggests that increasing the margin does not necessarily lead to better performance. On the contrary, a too large margin can make the optimization more difficult and, in some cases, can **reduce the quality of the learned representations**, confirming that the margin is a sensitive hyperparameter that must be carefully tuned.

Last Remark:

The ArcFace paper suggests using a margin value of at most  $m=0.5$ ; larger margins may make the training excessively difficult and prevent convergence. A certain consistency can be observed between the curve reported in the paper and the results obtained in our experiments. The left image refers to a training run with 2,000 classes, while the right one refers to a training run with 20,000 classes, both training with ArcFace and  $s=30$ .

In particular, the curve corresponding to the training with 2,000 classes and a margin between 0.4 and 0.6 indicates that the saturation of the probability curve should occur at approximately 40 – 45 degrees. This observation is consistent with the experimental results, where the model, in the configurations with  $s=30$  and  $s=50$ , was not able to produce more accurate predictions, unlike the cases in which  $s=15$  was used.

## Analysis with a Larger Number of Classes and Images per Class

In this experiment, we analyse the effect of increasing both the number of classes and the number of images per class.

As described previously, this experiment is performed using **6000 identities** and **150 images for each identity**, defining a richer and more challenging training configuration.

For this experiment, we choose to use **AdaCos**, since it avoids the need for manual tuning of the scale factor.

From the training results, it is possible to observe an **improvement of the validation performance** according to the adopted evaluation metric, together with an overall improvement in the shape of the similarity distributions.

In particular, it can be observed that:

- the distribution of **non-target predictions** becomes more concentrated around its mean and shows a **higher peak**;
- the average similarity of the **target predictions** appears to be **slightly higher**, but without a large increase.

From this configuration, it is still clear that **AdaCos mainly focuses on increasing the similarity between samples of the same identity**, even if this behaviour slightly reduces the emphasis on penalising incorrect matches.

# Model Evaluation

## Evaluation setup

For the final evaluation, I decided to compare the **AdaCos** model and the **ArcFace** model with a fixed scale factor  $s=50$  both trained using the same configuration with **3000 identities**. This choice allows a fair comparison and makes it possible to highlight the effect of the **scale factor and training philosophy** under the same training conditions. In addition, I also include the **AdaCos model trained with 6000 identities** in the evaluation. This allows us to highlight how increasing the size and diversity of the training set can improve the overall quality of the learned representations.

To evaluate models I implemented the following pipeline:

---

## Feature extraction

For all images in the evaluation set, embeddings are first extracted using the trained model and then normalised

---

## 1. Verification protocol (1 vs 1)

The verification task evaluates whether two images belong to the same identity.

All pairwise cosine similarities between embeddings are computed and divided into:

- **positive pairs** (same identity),
- **negative pairs** (different identities).

From these distributions, the following metric is computed:

- **TAR (True Acceptance Rate) at fixed FAR (False Acceptance Rate).**

For a given FAR value, a similarity threshold is estimated from the negative pairs, and the corresponding TAR is computed on the positive pairs.

This allows measuring how well the model separates genuine matches from impostors.

---

## 2. Identification protocol (1 vs N)

The identification task follows a standard **probe-gallery** setting.

For each identity:

- one image is randomly selected as **probe**,
- all remaining images are assigned to the **gallery**.

For every probe, cosine similarity is computed against all gallery samples.

The following metrics are then evaluated:

- **Rank-k accuracy** (e.g. Rank-1, Rank-5).

A prediction is considered correct at rank  $k$  if at least one gallery image of the correct identity appears among the top- $k$  most similar samples.

---

## 3. Robust evaluation over multiple runs

Since the probe-gallery split is random, both verification and identification experiments are repeated multiple times.

For each run:

- TAR at different FAR levels is computed for the verification task,
- Rank-k accuracy is computed for the identification task.

The final results are reported as:

- **mean and standard deviation** over multiple runs.

This provides a more reliable estimate of model performance.

---

## 4. Visual inspection of 1 vs N retrieval

For qualitative analysis, several random probe examples are selected.

For each probe:

- the top-5 most similar gallery images are displayed,
- correct matches and impostor matches are visually highlighted.

This visualization helps analysing typical successes and failure cases of the model.

---

## 5. Analysis of hardest negatives

To better understand the most critical errors made by the model, the pairs of images belonging to **different identities** with the highest cosine similarity are extracted.

These pairs represent the **hardest negative examples**, i.e. the most confusing impostor matches.

The corresponding images are visualised to inspect which visual patterns lead to ambiguity.

---

## 6. t-SNE visualisation of the embedding space

To analyse the global structure of the learned embeddings, a two-dimensional projection is computed using **t-SNE** on a subset of identities.

This allows visually inspecting:

- cluster compactness,
  - inter-class separation,
  - possible overlaps between identities.
- 

## 7. Similarity distribution and verification thresholds

Finally, the distributions of cosine similarities for:

- positive pairs,
- negative pairs,

are plotted together.

For several predefined FAR levels, the corresponding verification thresholds are computed and shown as vertical lines on the plot.

For each threshold, the associated TAR is also reported.

This analysis provides a direct visual interpretation of the verification performance and the amount of overlap between genuine and impostor distributions.

### **Note on the quality of the evaluation dataset**

During the analysis of the **hardest negative pairs**, I noticed a problem in **Test Dataset 1**.

In particular, when visualising the hardest negatives, some image pairs labelled as belonging to different identities actually appear to represent the same person assigned to different identity labels.

This indicates that a small portion of the dataset contains incorrect or inconsistent identity annotations.

Despite this issue, I decided to keep this test set in the evaluation.

The reason is that the problem seems to affect only a limited number of samples, and therefore



it is unlikely to significantly compromise the overall evaluation results and the general trends observed in the experiments.

## Final considerations

1. From a first qualitative and visual inspection, the models show a reasonable level of consistency in their predictions.  
Higher similarity scores are generally assigned to the correct identities, and when errors occur, they usually involve identities that share similar facial traits and visual characteristics with the target identity.
2. Test set 1 contains very few images per identity (most identities have only two samples).  
This makes the evaluation of **Rank-1 and Rank-5** accuracy more difficult.  
However, the similarity distributions appear better separated than those observed on the validation set, and the correct matches tend to be concentrated at higher similarity values.  
This behaviour is expected because, compared to the validation set, this test set includes fewer samples per identity and therefore less variability in pose, illumination and facial expression.  
For this reason, the matching task is simpler in terms of intra-class variability, even though the main challenge in this scenario is the **large number of non-target identities**.
3. **Test set 3** has a structure similar to the validation set, but with a smaller number of identities.  
At the same time, it provides a relatively large number of images per identity and the overall image quality is really good, this makes the task easier.  
As a consequence, both **Rank-1 and Rank-5** accuracies are higher than those obtained on the validation set.
4. Among the evaluated configurations, the **AdaCos model trained with 6000 identities** provides the best overall performance.
5. The most relevant observation concerns the comparison between AdaCos and ArcFace with a large scale factor. On one side, **AdaCos** can produce a large number of correct matches with a **relatively high similarity score**.  
However, this comes at the cost of giving **high similarity scores also to a noticeable number of wrong matches**.  
In other words, AdaCos tends to focus on learning facial patterns that increase intra-class similarity, but it penalizes **visually similar identities from different classes less strongly**. On the other side, **ArcFace** produces similarity scores that are, on average, **lower and more conservative**.  
At the same time, it seems to be **more effective in lowering the similarity of hard negatives**, creating a clearer separation between difficult classes. ArcFace therefore has a more cautious scoring behavior and appears more focused on discrimination, probably relying on simpler but still effective patterns to separate different

identities. The results on the two additional test sets clearly show the **trade-off between these two training philosophies**.

```
@title Test
import torch
import torch.nn.functional as F
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.cm as cm
from sklearn.manifold import TSNE
from tqdm.auto import tqdm
import random

=====
TAR/FAR
=====
def evaluate_tar_far(pos_sims, neg_sims, far_target=1e-3):
 neg_sims = np.sort(neg_sims)
 pos_sims = np.sort(pos_sims)
 num_neg = len(neg_sims)
 cutoff_idx = int((1.0 - far_target) * num_neg)
 cutoff_idx = min(cutoff_idx, num_neg - 1)
 threshold = neg_sims[cutoff_idx]
 num_pos = len(pos_sims)
 count_above_thresh = np.sum(pos_sims > threshold)
 tar = count_above_thresh / num_pos
 return tar, threshold

=====
CLASSE FACE EVALUATOR
=====
class FaceEvaluator:
 def __init__(self, model, loader, device='cuda'):
 self.model = model.to(device)
 self.loader = loader
 self.dataset = loader.dataset
 self.device = device
 self.embeddings = None
 self.labels = None
 self.indices_map = None

 def extract_features(self):
 """Estraggo feature una volta sola."""
 if self.embeddings is not None:
 return

 self.model.eval()
 print(">>> Estrazione Features in corso...")

 emb_list = []
```

```

label_list = []

with torch.no_grad():
 for imgs, labels in tqdm(self.loader, desc="Extracting"):
 imgs = imgs.to(self.device)
 emb = self.model(imgs)
 emb = F.normalize(emb, p=2, dim=1)
 emb_list.append(emb.cpu())
 label_list.append(labels.cpu())

self.embeddings = torch.cat(emb_list)
self.labels = torch.cat(label_list)
self.indices_map = torch.arange(len(self.labels))
print(f"Feature estratte: {self.embeddings.shape}\n")

def _get_image_for_plot(self, idx):
 img_tensor, _ = self.dataset[idx]
 img = img_tensor.permute(1, 2, 0).numpy()
 img = (img - img.min()) / (img.max() - img.min())
 return img

def run_robust_evaluation(self, num_runs=5, far_levels=[1e-2, 1e-3], ranks=[1, 5]):
 """
 Esegue test lvs1 e lvsN più volte, calcola Media/Std.
 """
 self.extract_features()
 print(f"\n{'='*20} EVALUATION ({num_runs} RUNS) {'='*20}")
 history_lvs1 = {far: [] for far in far_levels}
 history_lvsN = {rank: [] for rank in ranks}
 sim_matrix = torch.mm(self.embeddings, self.embeddings.t())
 labels_expand = self.labels.view(-1, 1)
 mask_pos = torch.eq(labels_expand, labels_expand.t())
 diag_mask = torch.eye(mask_pos.size(0), dtype=torch.bool)
 mask_pos = mask_pos & (~diag_mask)
 mask_neg = ~mask_pos & (~diag_mask)

 pos_sims_all = sim_matrix[mask_pos].numpy()
 neg_sims_all = sim_matrix[mask_neg].numpy()

 unique_labels = torch.unique(self.labels)
 class_indices = {lbl.item(): (self.labels ==
 lbl).nonzero(as_tuple=True)[0] for lbl in unique_labels}

 # --- LOOP sui run ---
 for run_idx in range(1, num_runs + 1):
 #print(f"\n--- RUN {run_idx}/{num_runs} ---")

 # --- 1. Calcolo lvs1 (Verification) ---

```

```

 if len(neg_sims_all) > 1_000_000:
 neg_sims_sample = np.random.choice(neg_sims_all,
1_000_000, replace=False)
 else:
 neg_sims_sample = neg_sims_all

 #print(f"[1vs1] Verification:")
 for far in far_levels:
 tar, _ = evaluate_tar_far(pos_sims_all,
neg_sims_sample, far)
 history_1vs1[far].append(tar)
 #print(f" FAR={far:<7} -> TAR: {tar*100:.2f}%")

 # --- 2. Calcolo 1vsN (Identification) ---
 probe_indices, gallery_indices = [], []

 for lbl, indices in class_indices.items():
 if len(indices) < 2:
 gallery_indices.extend(indices.tolist())
 continue
 indices = indices[torch.randperm(len(indices))]
 probe_indices.append(indices[0].item())
 gallery_indices.extend(indices[1:].tolist())

 probe_emb = self.embeddings[probe_indices]
 gallery_emb = self.embeddings[gallery_indices]
 probe_lbl = self.labels[probe_indices]
 gallery_lbl = self.labels[gallery_indices]

 sim_mat_1N = torch.mm(probe_emb, gallery_emb.t())
 max_k = max(ranks)

 run_correct = {k: 0 for k in ranks}
 for i in range(len(probe_emb)):
 true_lbl = probe_lbl[i].item()
 _, top_indices = torch.topk(sim_mat_1N[i], k=max_k)
 top_labels = gallery_lbl[top_indices].numpy()
 for k in ranks:
 if true_lbl in top_labels[:k]:
 run_correct[k] += 1

 #print(f"[1vsN] Identification:")
 for k in ranks:
 acc = run_correct[k] / len(probe_emb)
 history_1vsN[k].append(acc)
 #print(f" Rank-{k:<5} -> Acc: {acc*100:.2f}%")

 print("\n" + "="*60)
 print(f" FINAL SUMMARY (Mean ± Std Dev over {num_runs} runs)")
 print("="*60)

```

```

 print(f"{'METRIC':<20} | {'MEAN':<10} | {'STD DEV':<10} |
{'MIN-MAX'}")
 print("-" * 60)

 for far in far_levels:
 vals = np.array(history_lvs1[far]) * 100
 print(f"TAR @ FAR={far:<7} | {vals.mean():6.2f}% | ±
{vals.std():4.2f}% | [{vals.min():.2f} - {vals.max():.2f}]")

 print("-" * 60)

 for k in ranks:
 vals = np.array(history_lvsN[k]) * 100
 print(f"Rank-{k:<13} | {vals.mean():6.2f}% | ±
{vals.std():4.2f}% | [{vals.min():.2f} - {vals.max():.2f}]")
 print("-" * 60)

 return history_lvs1, history_lvsN

=====
TEST 1vsN SINGOLO CON VISUALIZZAZIONE MULTIPLA
=====
def test_identification_lvsN(self, ranks=[1, 5], num_examples=3):
 """
 Esegue un test 1vsN singolo e plotta 'num_examples' casi
 diversi di Probe.
 """
 self.extract_features()
 print(f"\n{'='*20} 2. IDENTIFICATION (SINGLE RUN) + VIS
{'='*20}")

 unique_labels = torch.unique(self.labels)
 class_indices = {lbl.item(): (self.labels ==
lbl).nonzero(as_tuple=True)[0] for lbl in unique_labels}

 probe_indices, gallery_indices = [], []

 # Split casuale
 for lbl, indices in class_indices.items():
 if len(indices) < 2:
 gallery_indices.extend(indices.tolist())
 continue
 indices = indices[torch.randperm(len(indices))]
 probe_indices.append(indices[0].item())
 gallery_indices.extend(indices[1:].tolist())

 probe_emb = self.embeddings[probe_indices]
 gallery_emb = self.embeddings[gallery_indices]
 probe_lbl = self.labels[probe_indices]
 gallery_lbl = self.labels[gallery_indices]

```

```

probe_global_idx = self.indices_map[probe_indices]
gallery_global_idx = self.indices_map[gallery_indices]

sim_mat = torch.mm(probe_emb, gallery_emb.t())

correct_at_k = {k: 0 for k in ranks}
max_k = max(ranks)
for i in range(len(probe_emb)):
 true_lbl = probe_lbl[i].item()
 _, top_indices = torch.topk(sim_mat[i], k=max_k)
 top_labels = gallery_lbl[top_indices].numpy()
 for k in ranks:
 if true_lbl in top_labels[:k]:
 correct_at_k[k] += 1

print("Performance di questa run (per riferimento):")
for k in ranks:
 print(f"Rank-{k}: {correct_at_k[k]/len(probe_emb)*100:.2f}
%)

print(f"\n>>> Visualizzazione di {num_examples} esempi di
ricerca lvsN...")

if len(probe_emb) < num_examples:
 random_idxes = range(len(probe_emb))
else:
 random_idxes = random.sample(range(len(probe_emb)),
num_examples)

for i, rnd_idx in enumerate(random_idxes):
 p_emb = probe_emb[rnd_idx]
 p_lbl = probe_lbl[rnd_idx].item()
 p_real_idx = probe_global_idx[rnd_idx].item()

 sims = torch.mm(p_emb.unsqueeze(0),
gallery_emb.t()).squeeze(0)
 top_vals, top_inds = torch.topk(sims, k=5)

 print(f"\nExample {i+1}/{num_examples} | Probe ID:
{p_lbl}")

 plt.figure(figsize=(15, 4))

 plt.subplot(1, 6, 1)
 plt.imshow(self.get_image_for_plot(p_real_idx))
 plt.title(f"PROBE\nID: {p_lbl}", color='blue',
fontweight='bold')
 plt.axis('off')

```

```

 for j, (score, g_idx) in enumerate(zip(top_vals,
top_inds)):
 g_lbl = gallery_lbl[g_idx].item()
 g_real_idx = gallery_global_idx[g_idx].item()

 plt.subplot(1, 6, j+2)
 plt.imshow(self._get_image_for_plot(g_real_idx))
 color = 'green' if g_lbl == p_lbl else 'red'
 plt.title(f"Rank {j+1}\nSim: {score:.3f}\nID:
{g_lbl}", color=color)
 plt.axis('off')

 plt.tight_layout()
 plt.show()

 def test_verification(self, far_levels=[1e-2, 1e-3, 1e-4]):
 self.extract_features()
 print(f"\n{'='*20} 1. VERIFICATION REPORT (SINGLE RUN)
{'='*20}")
 sim_matrix = torch.mm(self.embeddings, self.embeddings.t())
 labels_expand = self.labels.view(-1, 1)
 mask_pos = torch.eq(labels_expand, labels_expand.t())
 diag_mask = torch.eye(mask_pos.size(0), dtype=torch.bool)
 mask_pos = mask_pos & (~diag_mask)
 mask_neg = ~mask_pos & (~diag_mask)

 pos_sims = sim_matrix[mask_pos].numpy()
 neg_sims = sim_matrix[mask_neg].numpy()

 if len(neg_sims) > 1_000_000:
 neg_sims_sample = np.random.choice(neg_sims, 1_000_000,
replace=False)
 else:
 neg_sims_sample = neg_sims

 results = {}
 print(f"{'FAR':<15} | {'TAR':<15} | {'Threshold':<15}")
 print("-" * 55)
 for far_target in far_levels:
 tar, threshold = evaluate_tar_far(pos_sims,
neg_sims_sample, far_target)
 results[far_target] = threshold
 print(f"{'far_target':<15} | {'tar*100:6.2f}%
|
{threshold:.4f}")
 print("-" * 55)

 plt.figure(figsize=(10, 6))
 plt.hist(neg_sims_sample, bins=100, alpha=0.5, color='red',

```

```

label='Negatives', density=True)
 plt.hist(pos_sims, bins=100, alpha=0.5, color='green',
label='Positives', density=True)

 colors = ['blue', 'orange', 'purple']
 for i, far in enumerate(far_levels[:3]):
 t = results[far]
 plt.axvline(x=t, color=colors[i], linestyle='--',
label=f'FAR={far}'))
 plt.title("Cosine Similarity Distribution")
 plt.legend()
 plt.show()

def analyze_hardest_negatives(self, top_k=5):
 self.extract_features()
 print(f"\n{'='*20} 3. HARDEST NEGATIVES {'='*20}")
 sim_matrix = torch.mm(self.embeddings, self.embeddings.t())
 labels_expand = self.labels.view(-1, 1)
 mask_pos = torch.eq(labels_expand, labels_expand.t())
 sim_matrix[mask_pos] = -2.0
 mask_tri = torch.triu(torch.ones_like(sim_matrix),
diagonal=1).bool()
 sim_matrix[~mask_tri] = -2.0

 flat_sims = sim_matrix.view(-1)
 values, indices = torch.topk(flat_sims, k=top_k)

 fig, axes = plt.subplots(top_k, 2, figsize=(8, 3*top_k))
 if top_k == 1: axes = [axes]

 for i, (val, idx) in enumerate(zip(values, indices)):
 row = (idx // sim_matrix.size(1)).item()
 col = (idx % sim_matrix.size(1)).item()
 id1 = self.labels[row].item()
 id2 = self.labels[col].item()
 img1 = self._get_image_for_plot(row)
 img2 = self._get_image_for_plot(col)

 ax_left = axes[i][0] if top_k > 1 else axes[0]
 ax_left.imshow(img1)
 ax_left.set_title(f"ID: {id1}", color='red')
 ax_left.axis('off')

 ax_right = axes[i][1] if top_k > 1 else axes[1]
 ax_right.imshow(img2)
 ax_right.set_title(f"ID: {id2}", color='red')
 ax_right.axis('off')
 print(f"Pair {i+1}: Sim={val.item():.4f} | ID {id1} vs ID
{id2}")
 plt.suptitle(f"Hardest Negatives", fontsize=16)

```



```

plt.tight_layout()
plt.show()

def plot_tsne(self, num_classes=10):
 self.extract_features()
 print(f"\n{'='*20} 4. T-SNE VISUALIZATION {'='*20}")
 unique_lbls = torch.unique(self.labels).numpy()
 if len(unique_lbls) > num_classes:
 target_lbls = np.random.choice(unique_lbls, num_classes,
replace=False)
 else:
 target_lbls = unique_lbls
 indices = []
 for lbl in target_lbls:
 idx = (self.labels ==
torch.tensor(lbl)).nonzero(as_tuple=True)[0]
 indices.extend(idx.tolist())
 indices = torch.tensor(indices)
 subset_emb = self.embeddings[indices].numpy()
 subset_lbl = self.labels[indices].numpy()

 tsne = TSNE(n_components=2, random_state=42, perplexity=30,
init='pca', learning_rate='auto')
 emb_2d = tsne.fit_transform(subset_emb)

 plt.figure(figsize=(10, 8))
 colors = cm.rainbow(np.linspace(0, 1, len(target_lbls)))
 for i, lbl in enumerate(target_lbls):
 mask = (subset_lbl == lbl)
 plt.scatter(emb_2d[mask, 0], emb_2d[mask, 1],
color=colors[i], label=f'ID {lbl}', alpha=0.7)
 plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
 plt.title(f"t-SNE Projection ({num_classes} Identities)")
 plt.tight_layout()
 plt.show()

def plot_similarity_distributions(self, far_levels=[1e-2, 1e-3,
1e-4]):
 """
 Calcola e plotta le distribuzioni di similarità e mostra le
 linee verticali
 per diverse soglie di FAR (False Acceptance Rate).
 """
 self.extract_features()
 print(f"\n{'='*20} DISTRIBUTION PLOT (MULTI-FAR) {'='*20}")

 sim_matrix = torch.mm(self.embeddings, self.embeddings.t())

 labels_expand = self.labels.view(-1, 1)
 mask_pos = torch.eq(labels_expand, labels_expand.t())

```

```

diag_mask = torch.eye(mask_pos.size(0), dtype=torch.bool)
mask_pos = mask_pos & (~diag_mask)
mask_neg = ~mask_pos & (~diag_mask)

pos_sims = sim_matrix[mask_pos].numpy()
neg_sims = sim_matrix[mask_neg].numpy()

4. Downsampling
if len(neg_sims) > 1_000_000:
 neg_sims = np.random.choice(neg_sims, 1_000_000,
replace=False)

avg_pos = np.mean(pos_sims) if len(pos_sims) > 0 else 0
avg_neg = np.mean(neg_sims) if len(neg_sims) > 0 else 0

print(f"Media Sim Positivi: {avg_pos:.4f}")
print(f"Media Sim Negativi: {avg_neg:.4f}")
print("-" * 50)
print(f"{'FAR':<10} | {'TAR':<10} | {'THRESHOLD'}")
print("-" * 50)
plt.figure(figsize=(12, 6))
plt.hist(neg_sims, bins=150, alpha=0.5, color='red',
label='Negatives', density=True)
plt.hist(pos_sims, bins=150, alpha=0.5, color='green',
label='Positives', density=True)

colors = ['blue', 'darkorange', 'purple', 'cyan', 'black']

for i, far in enumerate(far_levels):
 tar, threshold = evaluate_tar_far(pos_sims, neg_sims,
far_target=far)
 print(f"{'far':<10} | {'tar*100:6.2f}% | {'threshold:.4f}")
 color_idx = i % len(colors)
 plt.axvline(x=threshold, color=colors[color_idx],
linestyle='--', linewidth=2,
label=f'FAR={far} (TAR={tar*100:.1f}%)')

 print("-" * 50)

plt.title(f"Similarity Distributions & Verification
Thresholds", fontsize=14)
plt.xlabel("Cosine Similarity")
plt.ylabel("Density")
plt.legend(loc='upper left')
plt.grid(True, alpha=0.3)
plt.xlim([-1, 1])
plt.tight_layout()
plt.show()

```

```

BASE_PATH = "/kaggle/input/"

files = {
 "adacos_v1": "adacos/other/default/1/best_model_backbone (6)
(1).pth",
 "adacos_6000": "adacos-6000/other/default/1/best_model_backbone4
(1).pth",
 "arcface_s50": "arcface-s50/other/default/1/best_model_backbone
(8).pth"
}

device = 'cuda' if torch.cuda.is_available() else 'cpu'

```

## ADACOS

```

@title validation set
print("\n--- Setup Model: AdaCos ---")
model_adacos_v1 = FaceModel(model_name='seresnet50', use_gdc=False)
path_1 = os.path.join(BASE_PATH, files["adacos_v1"])
load_checkpoint(model_adacos_v1, path_1, device)
evaluator = FaceEvaluator(model_adacos_v1, val_loader)
evaluator.run_robust_evaluation(num_runs=5)
evaluator.plot_similarity_distributions()
evaluator.analyze_hardest_negatives()
evaluator.test_identification_lvsN(ranks=[1, 5], num_examples=3)
evaluator.plot_tsne()
clean_gpu_memory(model_adacos_v1)
model_adacos_v1 = None

--- Setup Model: AdaCos ---
Caricamento pesi da:
/kaggle/input/adacos/other/default/1/best_model_backbone (6)
(1).pth...
>>> Pesi caricati con successo (Strict=True).
>>> Estrazione Features in corso...

{"model_id": "6eb4b0a2ce354b64b29b59c853c18b98", "version_major": 2, "version_minor": 0}

Feature estratte: torch.Size([17534, 512])

===== EVALUATION (5 RUNS) =====

=====
FINAL SUMMARY (Mean ± Std Dev over 5 runs)
=====
METRIC | MEAN | STD DEV | MIN-MAX

```

|                 |  |        |  |        |  |                 |
|-----------------|--|--------|--|--------|--|-----------------|
| TAR @ FAR=0.01  |  | 28.83% |  | ±0.05% |  | [28.78 - 28.91] |
| TAR @ FAR=0.001 |  | 9.29%  |  | ±0.08% |  | [9.16 - 9.38]   |

|        |  |        |  |        |  |                 |
|--------|--|--------|--|--------|--|-----------------|
| Rank-1 |  | 77.90% |  | ±1.52% |  | [76.19 - 80.00] |
| Rank-5 |  | 90.67% |  | ±1.75% |  | [88.57 - 92.38] |

# ===== DISTRIBUTION PLOT (MULTI-FAR)

Media Sim Positivi: 0.3891

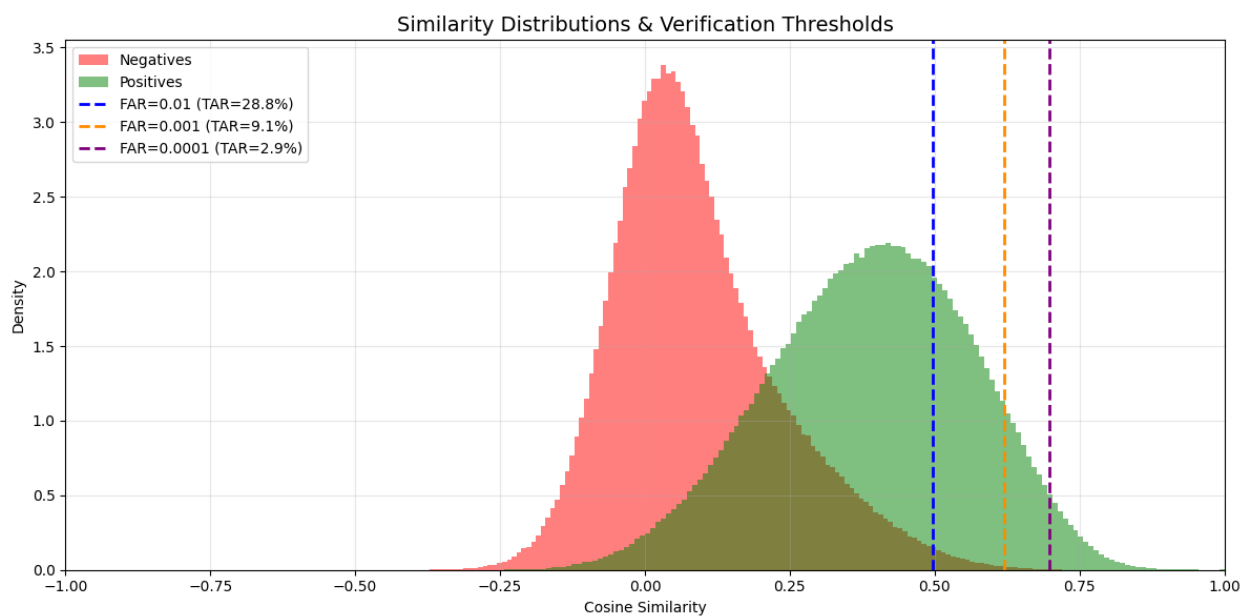
Media Sim Negativi: 0.0892

| FAR |  | TAR |  | THRESHOLD |
|-----|--|-----|--|-----------|
|-----|--|-----|--|-----------|

|      |  |        |  |        |
|------|--|--------|--|--------|
| 0.01 |  | 28.78% |  | 0.4960 |
|------|--|--------|--|--------|

|       |  |       |  |        |
|-------|--|-------|--|--------|
| 0.001 |  | 9.11% |  | 0.6203 |
|-------|--|-------|--|--------|

|        |  |       |  |        |
|--------|--|-------|--|--------|
| 0.0001 |  | 2.93% |  | 0.6979 |
|--------|--|-------|--|--------|



# ===== 3. HARDEST NEGATIVES =====

|                    |  |                 |
|--------------------|--|-----------------|
| Pair 1: Sim=0.9409 |  | ID 74 vs ID 87  |
| Pair 2: Sim=0.8729 |  | ID 76 vs ID 90  |
| Pair 3: Sim=0.8530 |  | ID 24 vs ID 35  |
| Pair 4: Sim=0.8508 |  | ID 35 vs ID 89  |
| Pair 5: Sim=0.8492 |  | ID 94 vs ID 103 |

## Hardest Negatives

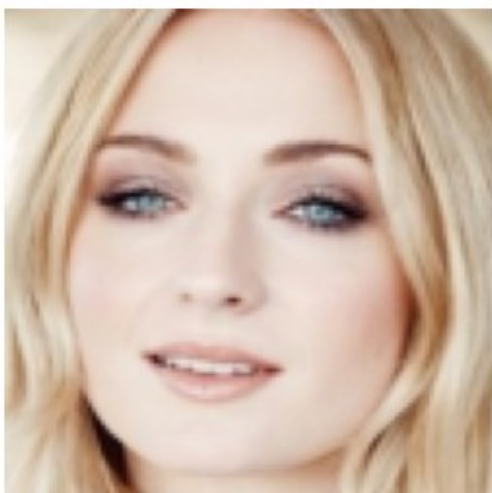
ID: 74



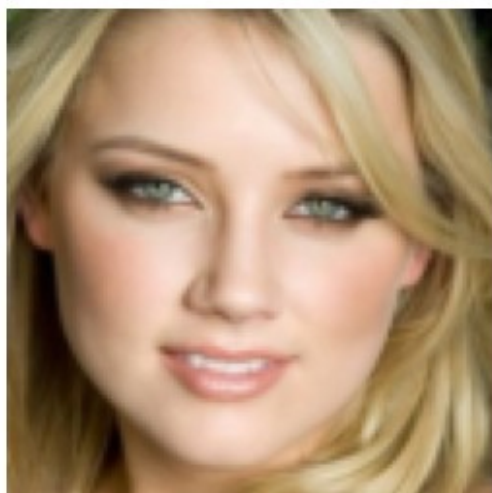
ID: 87



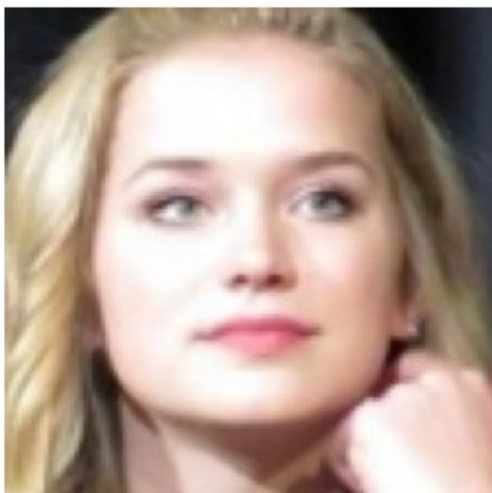
ID: 76



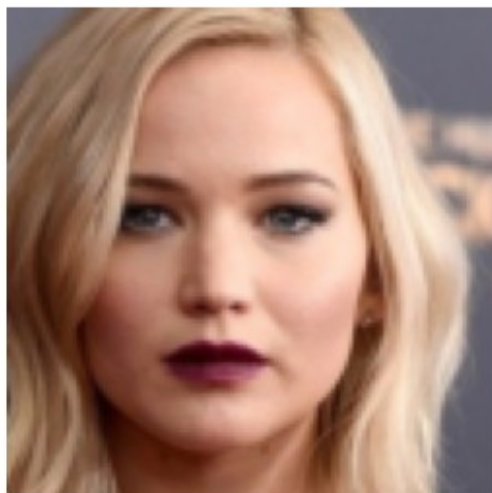
ID: 90



ID: 24



ID: 35



===== 2. IDENTIFICATION (SINGLE RUN) + VIS =====

Performance di questa run (per riferimento):

Rank-1: 81.90%

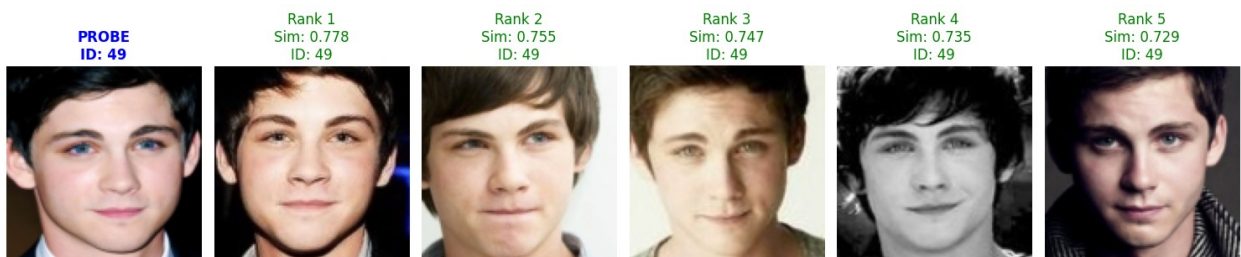
Rank-5: 94.29%

>>> Visualizzazione di 3 esempi di ricerca 1vsN...

Example 1/3 | Probe ID: 18



Example 2/3 | Probe ID: 49

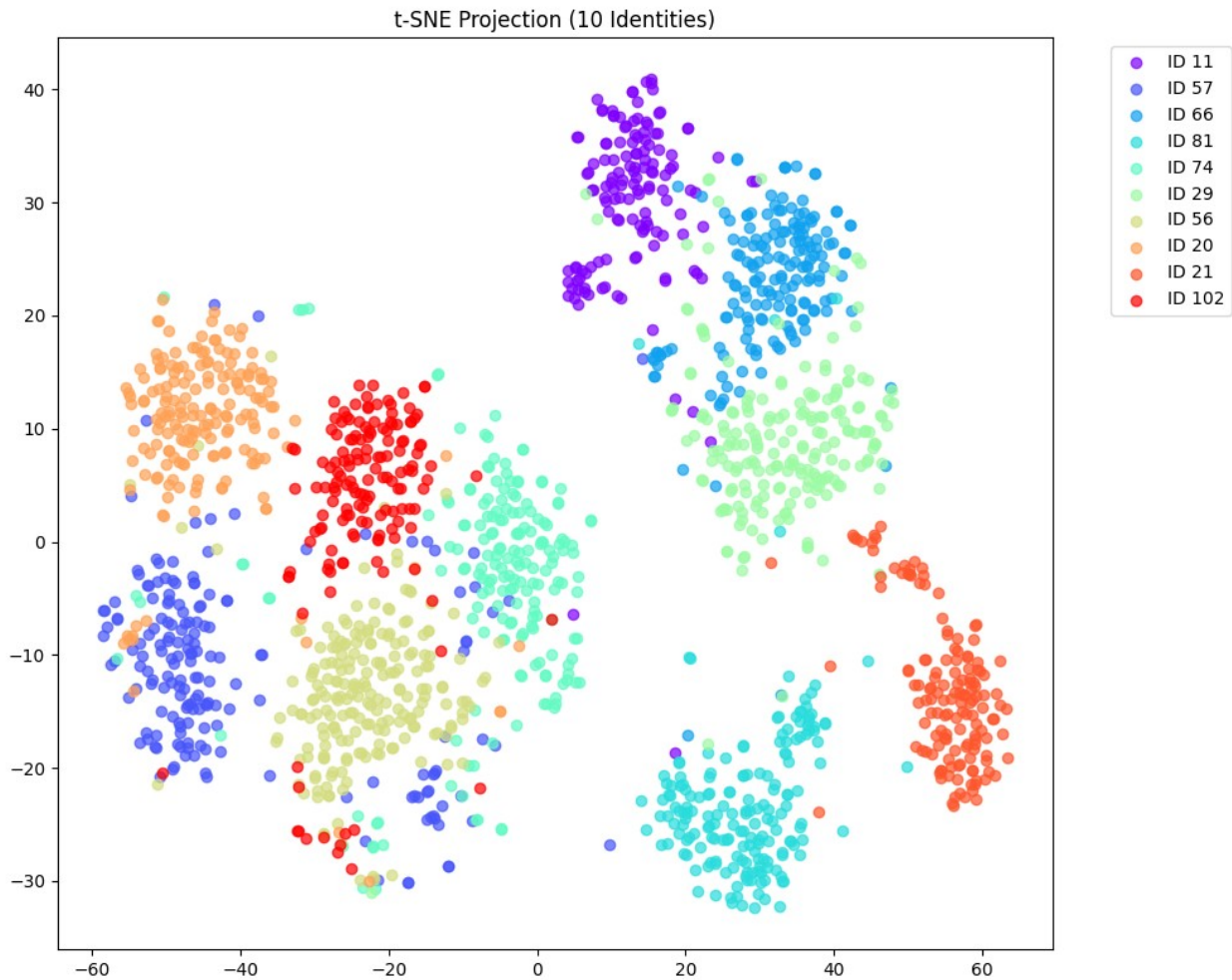


Example 3/3 | Probe ID: 17



===== 4. T-SNE VISUALIZATION =====





```
>>> GPU Memory Cleaned.
```

```
@title Test set 1
print("\n--- Setup Model: AdaCos ---")
model_adacos_v1 = FaceModel(model_name='seresnet50', use_gdc=False)
path_1 = os.path.join(BASE_PATH, files["adacos_v1"])
load_checkpoint(model_adacos_v1, path_1, device)

evaluator = FaceEvaluator(model_adacos_v1, test_loader_1)
evaluator.run_robust_evaluation(num_runs=5)
evaluator.plot_similarity_distributions()
evaluator.analyze_hardest_negatives()
evaluator.test_identification_1vsN(ranks=[1, 5], num_examples=3)
#evaluator.plot_tsne()

clean_gpu_memory(model_adacos_v1)
model_adacos_v1 = None
```

```
--- Setup Model: AdaCos ---
```

```
Caricamento pesi da:
/kaggle/input/adacos/other/default/1/best_model_backbone (6)
(1).pth...
>>> Pesi caricati con successo (Strict=True).
>>> Estrazione Features in corso...

{"model_id":"e67e46c749e9457992fef329a45c245d","version_major":2,"version_minor":0}

Feature estratte: torch.Size([6107, 512])
```

===== EVALUATION (5 RUNS) =====

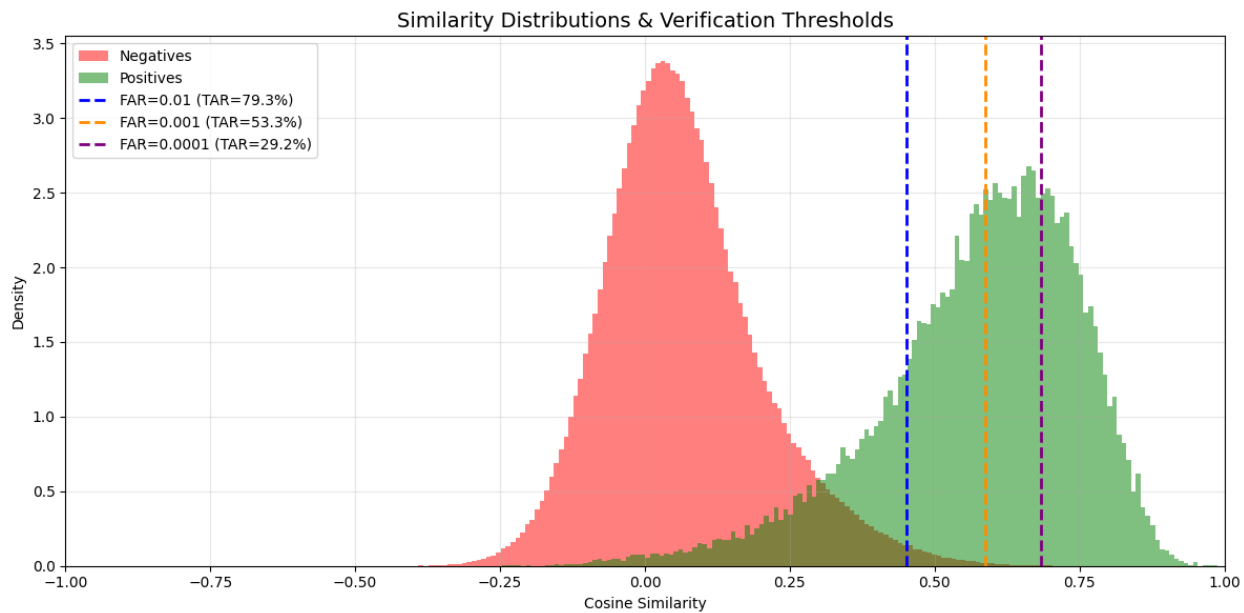
| FINAL SUMMARY (Mean ± Std Dev over 5 runs) |        |         |                 |
|--------------------------------------------|--------|---------|-----------------|
| METRIC                                     | MEAN   | STD DEV | MIN-MAX         |
| TAR @ FAR=0.01                             | 79.38% | ±0.04%  | [79.34 - 79.43] |
| TAR @ FAR=0.001                            | 53.27% | ±0.32%  | [52.77 - 53.65] |
| Rank-1                                     | 62.84% | ±0.54%  | [61.93 - 63.60] |
| Rank-5                                     | 77.37% | ±0.32%  | [76.89 - 77.72] |

===== DISTRIBUTION PLOT (MULTI-FAR) =====

Media Sim Positivi: 0.5749  
Media Sim Negativi: 0.0657

| FAR    | TAR    | THRESHOLD |
|--------|--------|-----------|
| 0.01   | 79.28% | 0.4509    |
| 0.001  | 53.34% | 0.5875    |
| 0.0001 | 29.20% | 0.6832    |





```
===== 3. HARDEST NEGATIVES =====
Pair 1: Sim=0.9010 | ID 178 vs ID 580
Pair 2: Sim=0.8708 | ID 831 vs ID 1214
Pair 3: Sim=0.8528 | ID 831 vs ID 1214
Pair 4: Sim=0.8357 | ID 194 vs ID 916
Pair 5: Sim=0.8340 | ID 194 vs ID 647
```

## Hardest Negatives

ID: 178



ID: 580



ID: 831



ID: 1214



ID: 831



ID: 1214



===== 2. IDENTIFICATION (SINGLE RUN) + VIS  
=====

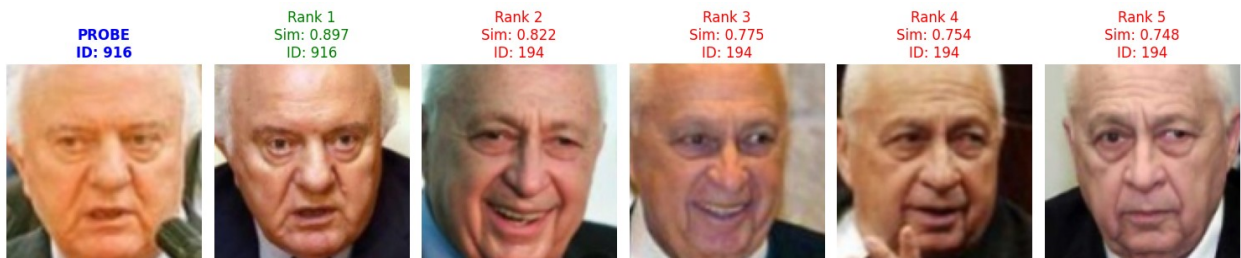
Performance di questa run (per riferimento):

Rank-1: 63.22%

Rank-5: 77.19%

>>> Visualizzazione di 3 esempi di ricerca 1vsN...

Example 1/3 | Probe ID: 916



Example 2/3 | Probe ID: 445



Example 3/3 | Probe ID: 515



>>> GPU Memory Cleaned.

```
@title Test set 2
print("\n--- Setup Model: AdaCos ---")
model_adacos_v1 = FaceModel(model_name='seresnet50', use_gdc=False)
path_1 = os.path.join(BASE_PATH, files["adacos_v1"])
```

```

load_checkpoint(model_adacos_v1, path_1, device)

evaluator = FaceEvaluator(model_adacos_v1, test_loader_2)
evaluator.run_robust_evaluation(num_runs=5)
evaluator.plot_similarity_distributions()
evaluator.analyze_hardest_negatives()
evaluator.test_identification_lvsN(ranks=[1, 5], num_examples=3)
evaluator.plot_tsne()
clean_gpu_memory(model_adacos_v1)
model_adacos_v1 = None

--- Setup Model: AdaCos ---
Caricamento pesi da:
/kaggle/input/adacos/other/default/1/best_model_backbone (6)
(1).pth...
>>> Pesi caricati con successo (Strict=True).
>>> Estrazione Features in corso...

{"model_id": "16a6f7352ab647dba06cc0ed49512434", "version_major": 2, "version_minor": 0}

Feature estratte: torch.Size([2562, 512])

===== EVALUATION (5 RUNS) =====

=====
FINAL SUMMARY (Mean ± Std Dev over 5 runs)
=====

```

| METRIC          | MEAN    | STD DEV | MIN-MAX           |
|-----------------|---------|---------|-------------------|
| TAR @ FAR=0.01  | 41.34%  | ±0.12%  | [41.16 - 41.49]   |
| TAR @ FAR=0.001 | 17.14%  | ±0.28%  | [16.76 - 17.58]   |
| Rank-1          | 97.42%  | ±2.41%  | [93.55 - 100.00]  |
| Rank-5          | 100.00% | ±0.00%  | [100.00 - 100.00] |

```

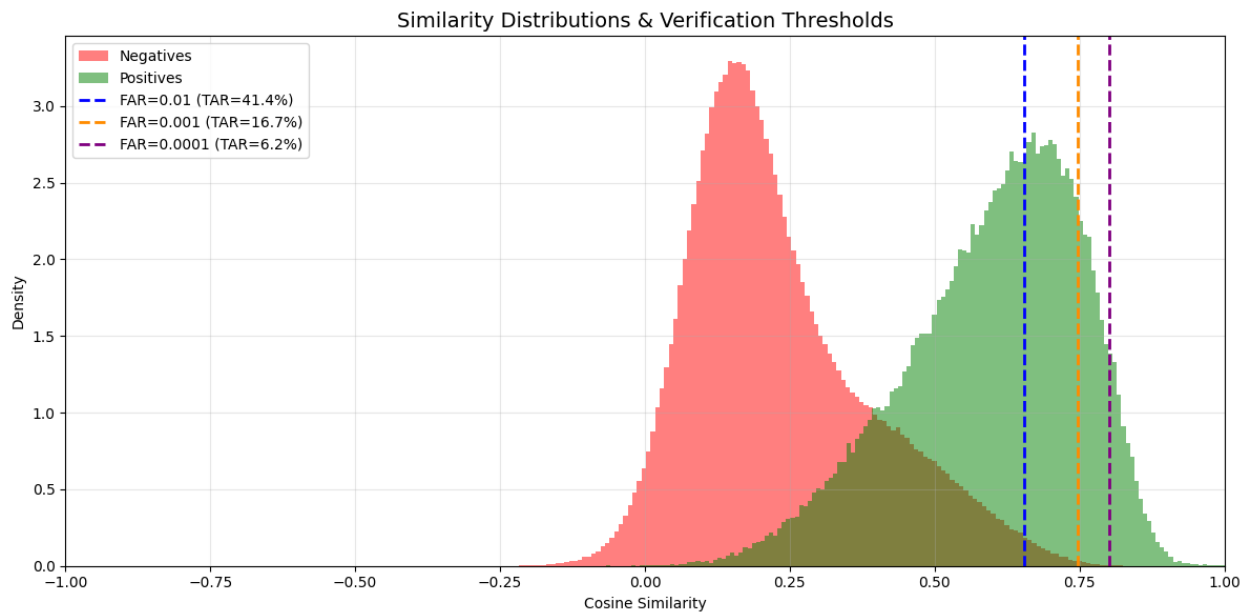
===== DISTRIBUTION PLOT (MULTI-FAR)
=====
Media Sim Positivi: 0.5994
Media Sim Negativi: 0.2314

```

| FAR    | TAR    | THRESHOLD |
|--------|--------|-----------|
| 0.01   | 41.36% | 0.6541    |
| 0.001  | 16.74% | 0.7461    |
| 0.0001 | 6.17%  | 0.8016    |

```

```



```
===== 3. HARDEST NEGATIVES =====
Pair 1: Sim=0.8926 | ID 1455 vs ID 1457
Pair 2: Sim=0.8808 | ID 1451 vs ID 1448
Pair 3: Sim=0.8650 | ID 1447 vs ID 1431
Pair 4: Sim=0.8634 | ID 1448 vs ID 1451
Pair 5: Sim=0.8631 | ID 1448 vs ID 1451
```

## Hardest Negatives

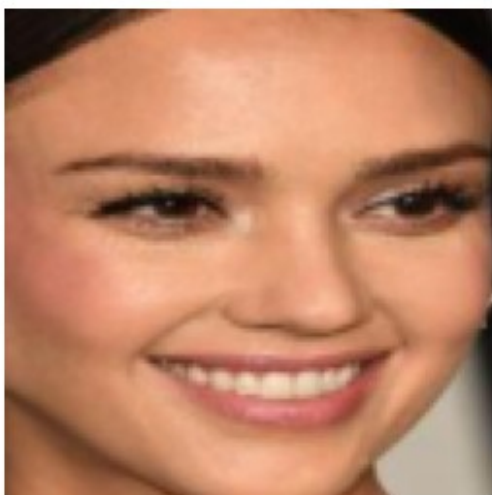
ID: 1455



ID: 1457



ID: 1451



ID: 1448



ID: 1447



ID: 1431





===== 2. IDENTIFICATION (SINGLE RUN) + VIS  
=====

Performance di questa run (per riferimento):

Rank-1: 96.77%

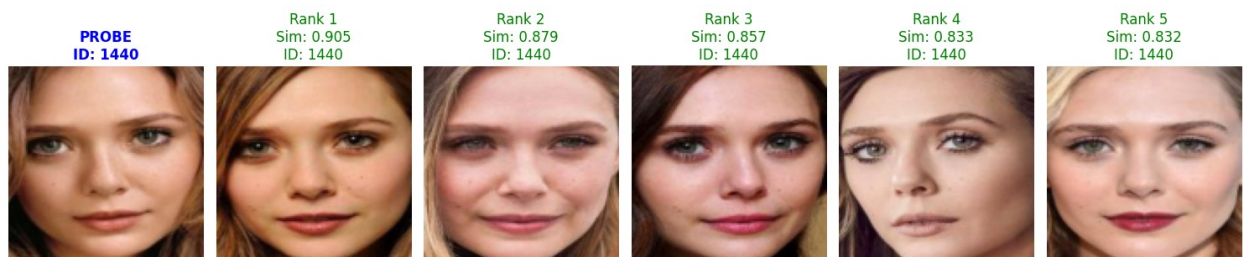
Rank-5: 100.00%

>>> Visualizzazione di 3 esempi di ricerca 1vsN...

Example 1/3 | Probe ID: 1447



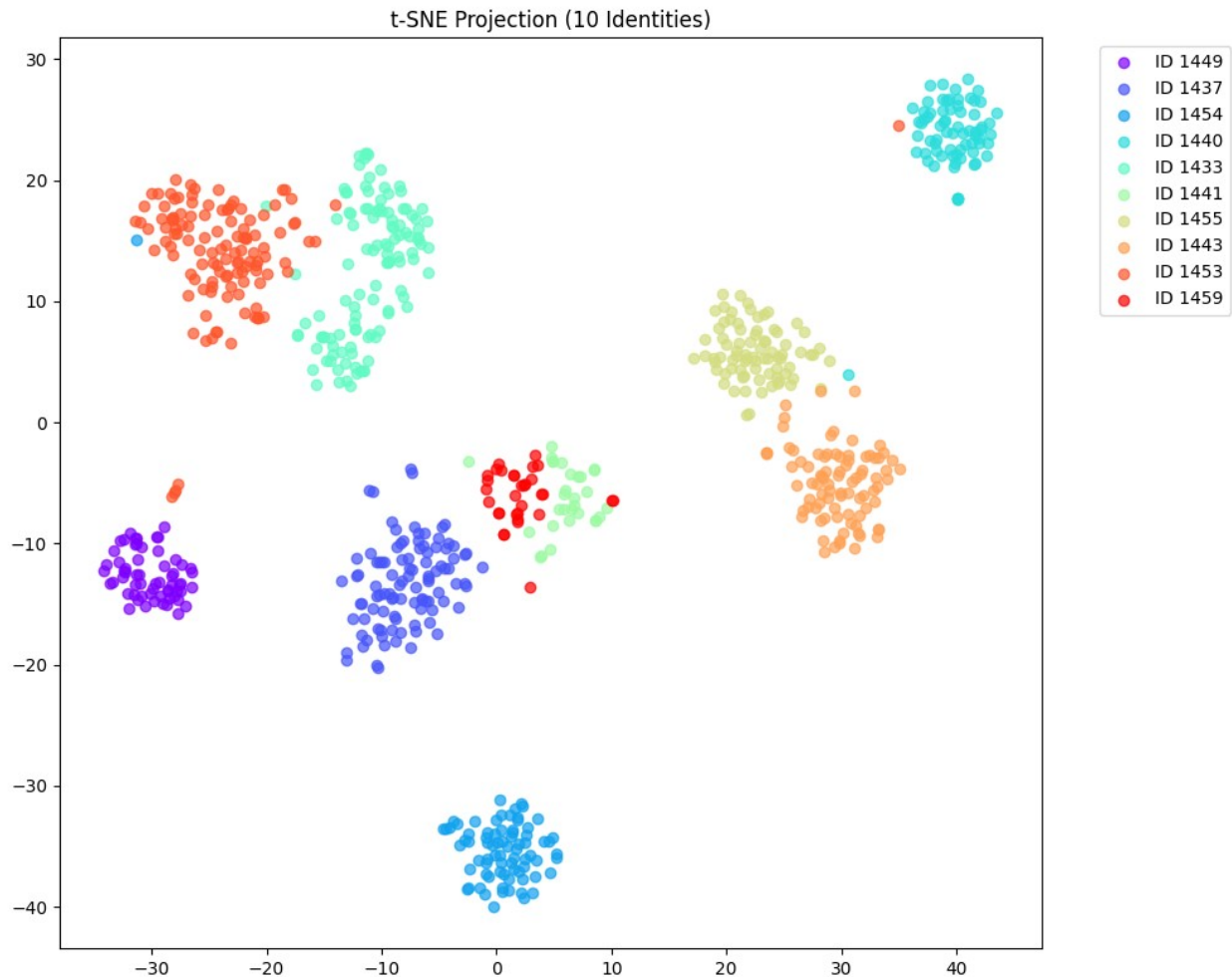
Example 2/3 | Probe ID: 1440



Example 3/3 | Probe ID: 1436



===== 4. T-SNE VISUALIZATION =====



```
>>> GPU Memory Cleaned.
```

## ADACOS 6000

```
@title validation set
print("\n--- Setup Model: AdaCos 6000 ---")
model_adacos_v1 = FaceModel(model_name='seresnet50', use_gdc=False)
path_1 = os.path.join(BASE_PATH, files["adacos_6000"])
load_checkpoint(model_adacos_v1, path_1, device)
evaluator = FaceEvaluator(model_adacos_v1, val_loader)
evaluator.run_robust_evaluation(num_runs=5)
evaluator.plot_similarity_distributions()
evaluator.analyze_hardest_negatives()
evaluator.test_identification_lvsN(ranks=[1, 5], num_examples=3)
evaluator.plot_tsne()

clean_gpu_memory(model_adacos_v1)
model_adacos_v1 = None
```



--- Setup Model: AdaCos 6000 ---

Caricamento pesi da:

/kaggle/input/adacos-6000/other/default/1/best\_model\_backbone4  
(1).pth...

>>> Pesi caricati con successo (Strict=True).

>>> Estrazione Features in corso...

```
{"model_id": "5a3476f036cf42138b1a5076cce9faf2", "version_major": 2, "version_minor": 0}
```

Feature estratte: torch.Size([17534, 512])

===== EVALUATION (5 RUNS) =====

=====

FINAL SUMMARY (Mean  $\pm$  Std Dev over 5 runs)

=====

| METRIC          | MEAN   | STD DEV      | MIN-MAX         |
|-----------------|--------|--------------|-----------------|
| TAR @ FAR=0.01  | 41.62% | $\pm 0.10\%$ | [41.50 - 41.71] |
| TAR @ FAR=0.001 | 15.98% | $\pm 0.16\%$ | [15.73 - 16.15] |
| Rank-1          | 84.76% | $\pm 1.90\%$ | [81.90 - 87.62] |
| Rank-5          | 93.14% | $\pm 1.94\%$ | [89.52 - 95.24] |

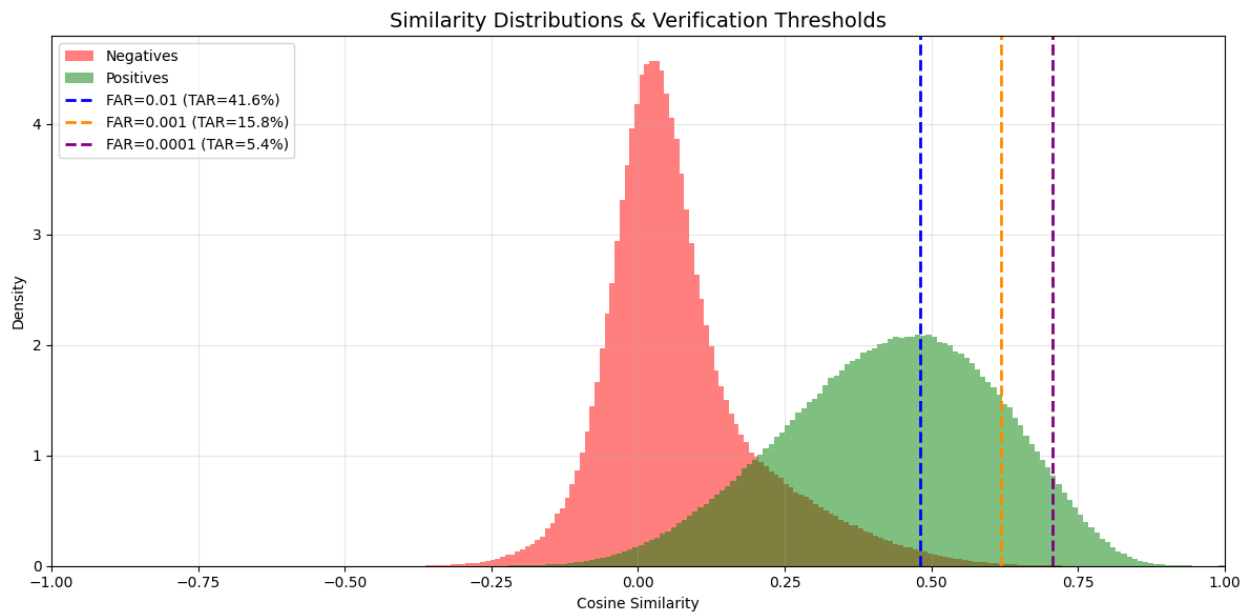
===== DISTRIBUTION PLOT (MULTI-FAR) =====

=====

Media Sim Positivi: 0.4306

Media Sim Negativi: 0.0706

| FAR    | TAR    | THRESHOLD |
|--------|--------|-----------|
| 0.01   | 41.62% | 0.4815    |
| 0.001  | 15.84% | 0.6183    |
| 0.0001 | 5.36%  | 0.7067    |



===== 3. HARDEST NEGATIVES =====

|         |            |  |                 |
|---------|------------|--|-----------------|
| Pair 1: | Sim=0.9055 |  | ID 74 vs ID 87  |
| Pair 2: | Sim=0.8806 |  | ID 50 vs ID 68  |
| Pair 3: | Sim=0.8768 |  | ID 24 vs ID 35  |
| Pair 4: | Sim=0.8765 |  | ID 50 vs ID 103 |
| Pair 5: | Sim=0.8663 |  | ID 76 vs ID 103 |

## Hardest Negatives

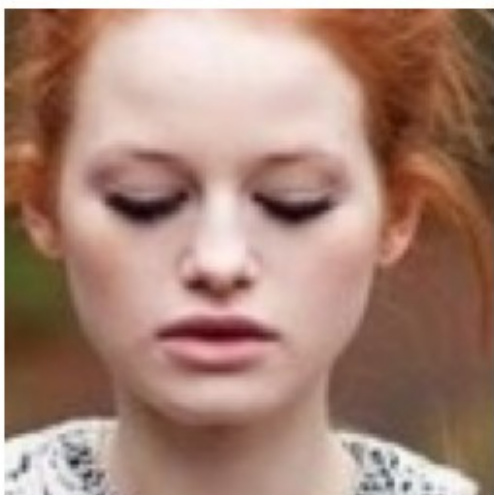
ID: 74



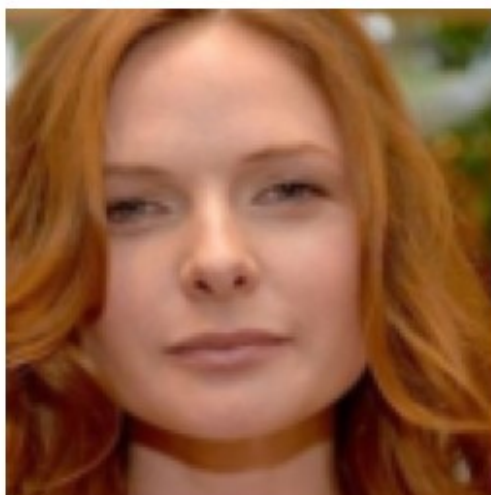
ID: 87



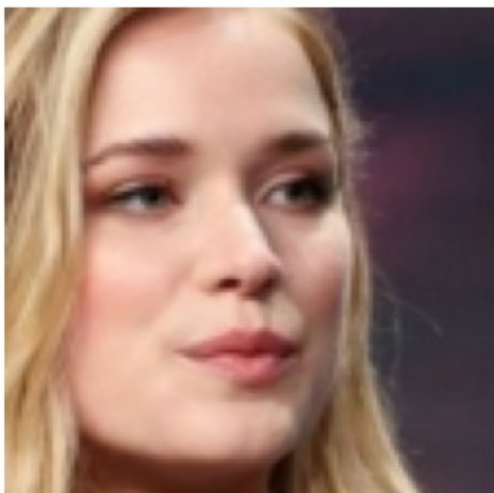
ID: 50



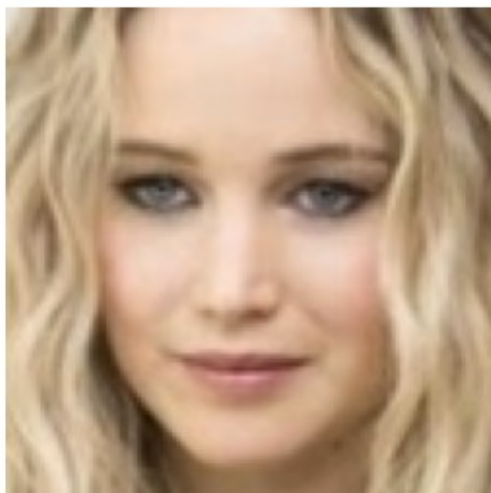
ID: 68



ID: 24



ID: 35



===== 2. IDENTIFICATION (SINGLE RUN) + VIS =====

Performance di questa run (per riferimento):

Rank-1: 82.86%

Rank-5: 90.48%

>>> Visualizzazione di 3 esempi di ricerca 1vsN...

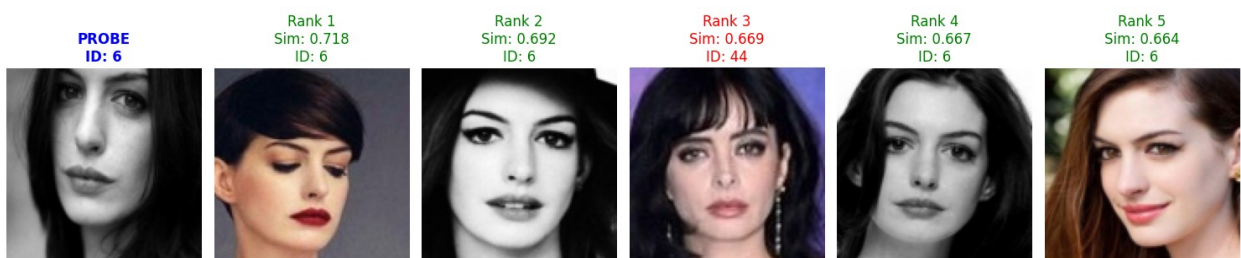
Example 1/3 | Probe ID: 73



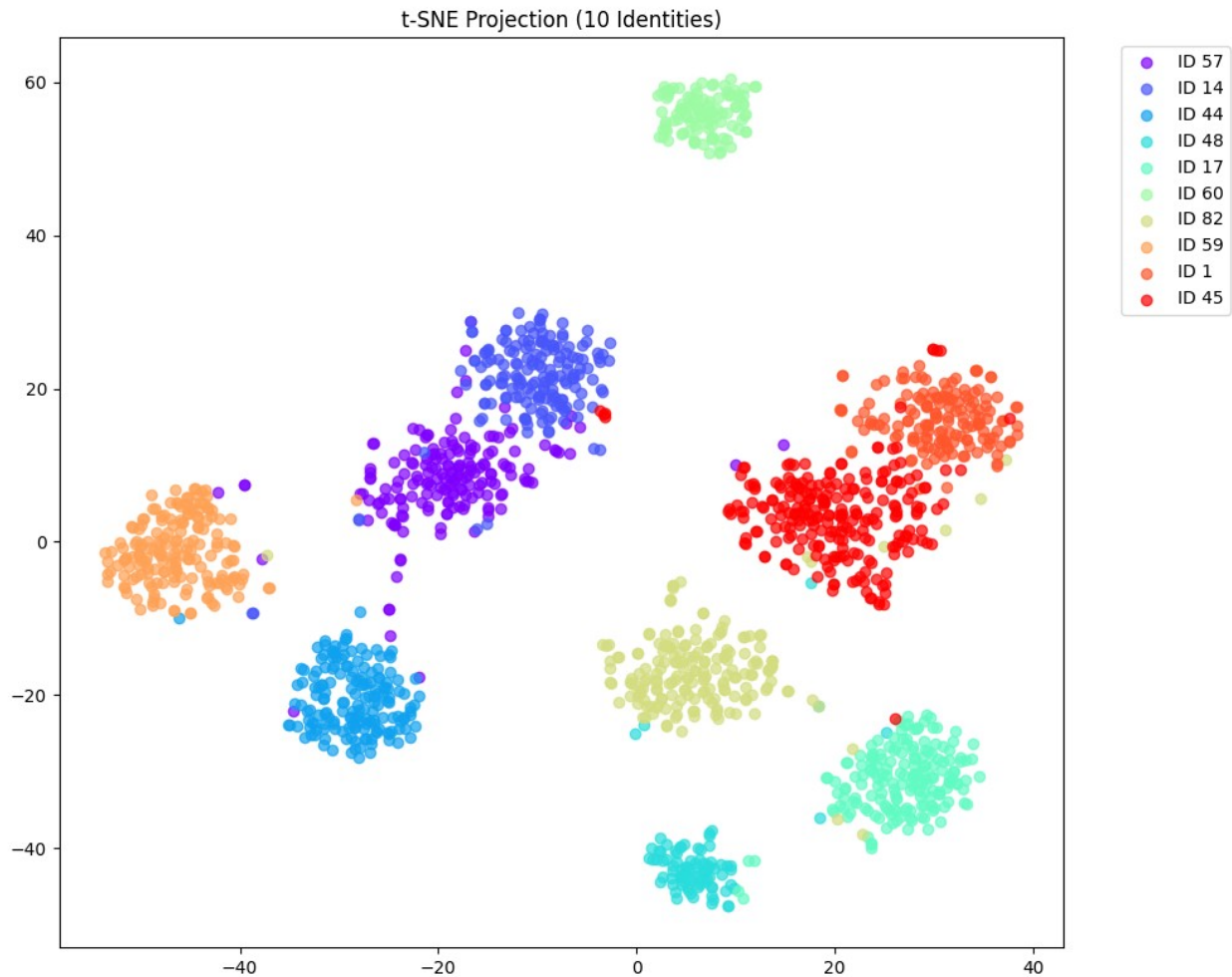
Example 2/3 | Probe ID: 3



Example 3/3 | Probe ID: 6



===== 4. T-SNE VISUALIZATION =====



```
>>> GPU Memory Cleaned.
```

```
@title Test set 1
print("\n--- Setup Model: AdaCos 6000 ---")
model_adacos_v1 = FaceModel(model_name='seresnet50', use_gdc=False)
path_1 = os.path.join(BASE_PATH, files["adacos_6000"])
load_checkpoint(model_adacos_v1, path_1, device)

evaluator = FaceEvaluator(model_adacos_v1, test_loader_1)
evaluator.run_robust_evaluation(num_runs=5)

evaluator.plot_similarity_distributions()
evaluator.analyze_hardest_negatives()
evaluator.test_identification_lvsN(ranks=[1, 5], num_examples=3)
#evaluator.plot_tsne()

clean_gpu_memory(model_adacos_v1)
model_adacos_v1 = None
```

--- Setup Model: AdaCos 6000 ---

Caricamento pesi da:

/kaggle/input/adacos-6000/other/default/1/best\_model\_backbone4  
(1).pth...

>>> Pesi caricati con successo (Strict=True).

>>> Estrazione Features in corso...

```
{"model_id": "54c61333995d40db9f553d902db75735", "version_major": 2, "version_minor": 0}
```

Feature estratte: torch.Size([6107, 512])

===== EVALUATION (5 RUNS) =====

=====

FINAL SUMMARY (Mean  $\pm$  Std Dev over 5 runs)

=====

| METRIC          | MEAN   | STD DEV      | MIN-MAX         |
|-----------------|--------|--------------|-----------------|
| TAR @ FAR=0.01  | 90.35% | $\pm 0.04\%$ | [90.28 - 90.39] |
| TAR @ FAR=0.001 | 69.83% | $\pm 0.30\%$ | [69.47 - 70.26] |
| Rank-1          | 75.08% | $\pm 0.71\%$ | [74.09 - 76.28] |
| Rank-5          | 86.44% | $\pm 0.75\%$ | [85.57 - 87.76] |

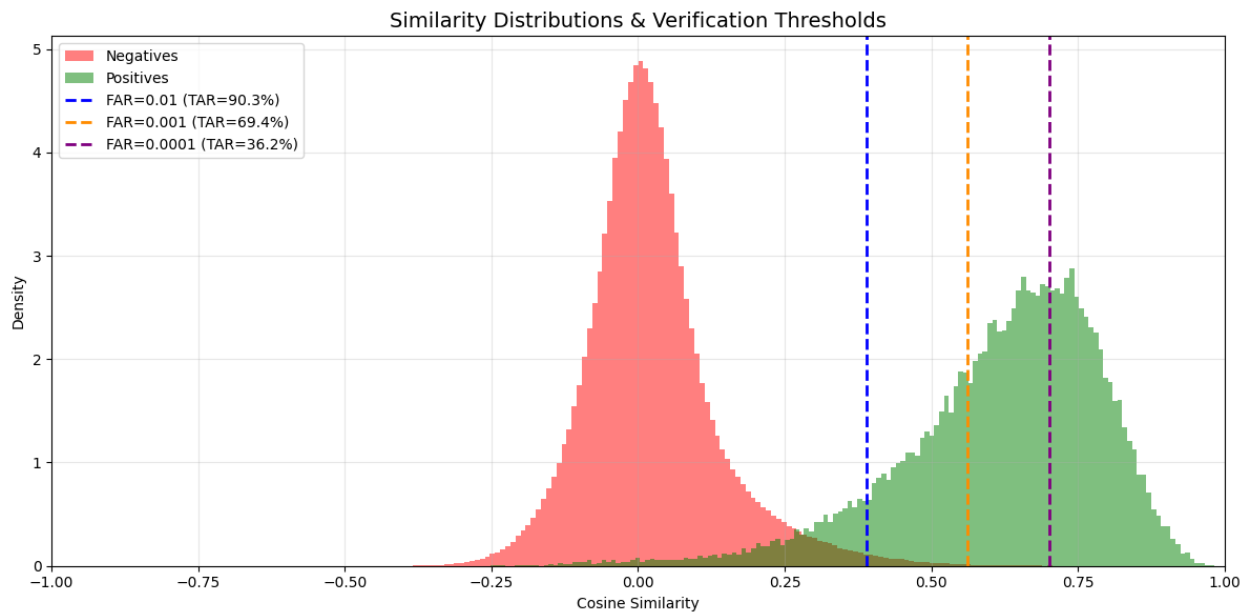
===== DISTRIBUTION PLOT (MULTI-FAR) =====

=====

Media Sim Positivi: 0.6204

Media Sim Negativi: 0.0233

| FAR    | TAR    | THRESHOLD |
|--------|--------|-----------|
| 0.01   | 90.33% | 0.3893    |
| 0.001  | 69.43% | 0.5621    |
| 0.0001 | 36.25% | 0.7009    |



```
===== 3. HARDEST NEGATIVES =====
Pair 1: Sim=0.9141 | ID 178 vs ID 580
Pair 2: Sim=0.8992 | ID 802 vs ID 1100
Pair 3: Sim=0.8826 | ID 802 vs ID 1100
Pair 4: Sim=0.8780 | ID 679 vs ID 1100
Pair 5: Sim=0.8685 | ID 802 vs ID 1100
```



## Hardest Negatives

ID: 178



ID: 580



ID: 802



ID: 1100



ID: 802



ID: 1100





===== 2. IDENTIFICATION (SINGLE RUN) + VIS  
=====

Performance di questa run (per riferimento):

Rank-1: 74.92%

Rank-5: 87.54%

>>> Visualizzazione di 3 esempi di ricerca 1vsN...

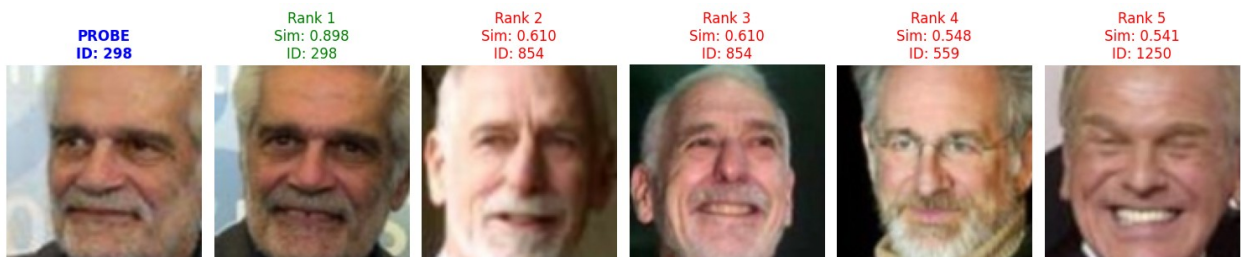
Example 1/3 | Probe ID: 1268



Example 2/3 | Probe ID: 376



Example 3/3 | Probe ID: 298



>>> GPU Memory Cleaned.

```
@title Test set 2
print("\n--- Setup Model: AdaCos 6000 ---")
model_adacos_v1 = FaceModel(model_name='seresnet50', use_gdc=False)
path_1 = os.path.join(BASE_PATH, files["adacos_6000"])
```

```

load_checkpoint(model_adacos_v1, path_1, device)

evaluator = FaceEvaluator(model_adacos_v1, test_loader_2)
evaluator.run_robust_evaluation(num_runs=5)
evaluator.plot_similarity_distributions()
evaluator.analyze_hardest_negatives()
evaluator.test_identification_lvsN(ranks=[1, 5], num_examples=3)
evaluator.plot_tsne()
clean_gpu_memory(model_adacos_v1)
model_adacos_v1 = None

--- Setup Model: AdaCos 6000 ---
Caricamento pesi da:
/kaggle/input/adacos-6000/other/default/1/best_model_backbone4
(1).pth...
>>> Pesi caricati con successo (Strict=True).
>>> Estrazione Features in corso...

{"model_id": "12ac7c544d94469793fffd0a430d66c1", "version_major": 2, "version_minor": 0}

Feature estratte: torch.Size([2562, 512])

===== EVALUATION (5 RUNS) =====

=====
FINAL SUMMARY (Mean ± Std Dev over 5 runs)
=====

```

| METRIC          | MEAN   | STD DEV | MIN-MAX          |
|-----------------|--------|---------|------------------|
| TAR @ FAR=0.01  | 60.00% | ±0.10%  | [59.80 - 60.09]  |
| TAR @ FAR=0.001 | 30.87% | ±0.14%  | [30.66 - 31.03]  |
| Rank-1          | 95.48% | ±1.58%  | [93.55 - 96.77]  |
| Rank-5          | 98.06% | ±1.58%  | [96.77 - 100.00] |

```

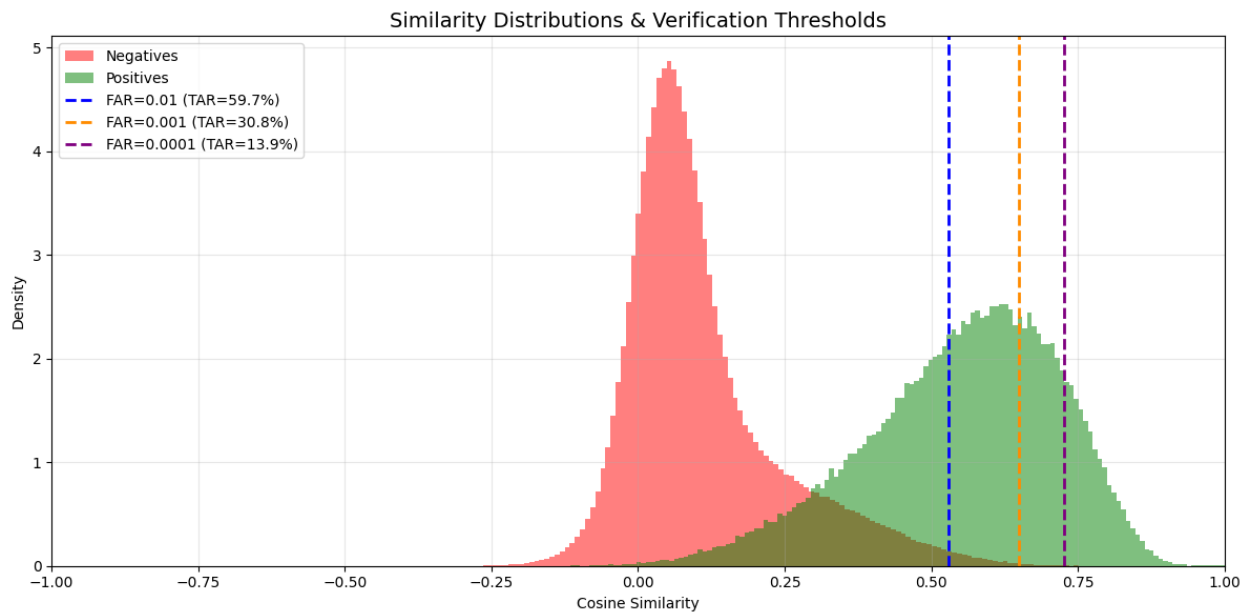
===== DISTRIBUTION PLOT (MULTI-FAR)
=====
Media Sim Positivi: 0.5519
Media Sim Negativi: 0.1109

```

| FAR    | TAR    | THRESHOLD |
|--------|--------|-----------|
| 0.01   | 59.75% | 0.5291    |
| 0.001  | 30.85% | 0.6494    |
| 0.0001 | 13.94% | 0.7261    |

```

```



```
===== 3. HARDEST NEGATIVES =====
Pair 1: Sim=0.8237 | ID 1438 vs ID 1440
Pair 2: Sim=0.8216 | ID 1452 vs ID 1450
Pair 3: Sim=0.8205 | ID 1431 vs ID 1438
Pair 4: Sim=0.8147 | ID 1438 vs ID 1440
Pair 5: Sim=0.8134 | ID 1452 vs ID 1450
```

## Hardest Negatives

ID: 1438



ID: 1440



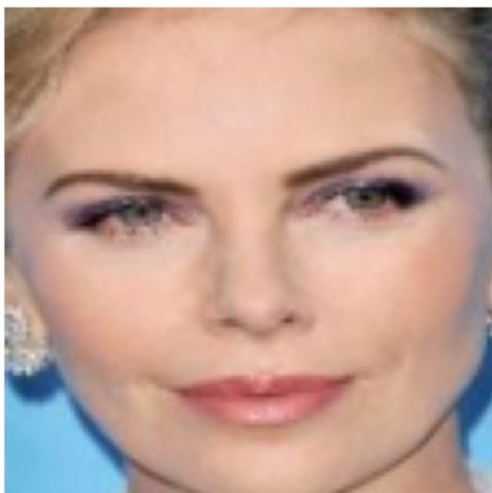
ID: 1452



ID: 1450



ID: 1431



ID: 1438



===== 2. IDENTIFICATION (SINGLE RUN) + VIS  
=====

Performance di questa run (per riferimento):

Rank-1: 100.00%

Rank-5: 100.00%

>>> Visualizzazione di 3 esempi di ricerca 1vsN...

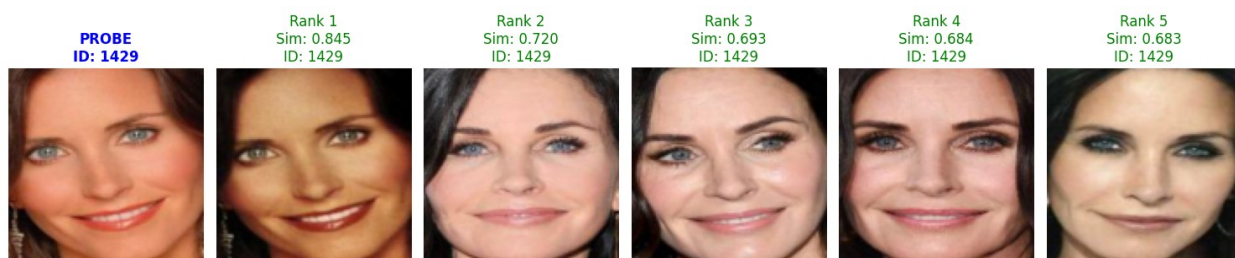
Example 1/3 | Probe ID: 1436



Example 2/3 | Probe ID: 1432

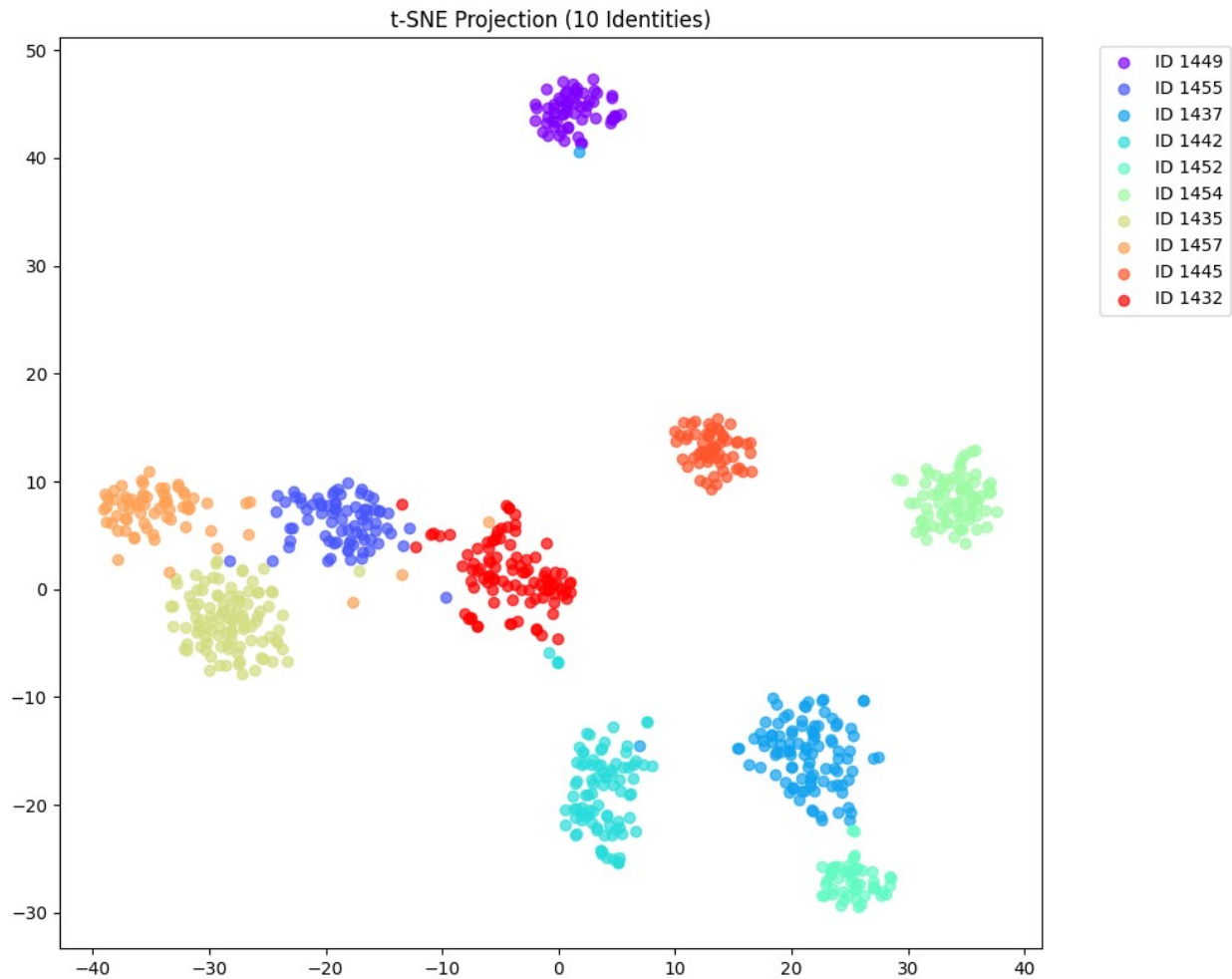


Example 3/3 | Probe ID: 1429



===== 4. T-SNE VISUALIZATION =====





```
>>> GPU Memory Cleaned.
```

## ARCFACE

```
@title validation set
print("\n--- Setup Model: Arcface ---")
model_adacos_v1 = FaceModel(model_name='seresnet50', use_gdc=False)
path_1 = os.path.join(BASE_PATH, files["arcface_s50"])
load_checkpoint(model_adacos_v1, path_1, device)

evaluator = FaceEvaluator(model_adacos_v1, val_loader)
evaluator.run_robust_evaluation(num_runs=5)
evaluator.plot_similarity_distributions()
evaluator.analyze_hardest_negatives()
evaluator.test_identification_lvsN(ranks=[1, 5], num_examples=3)
evaluator.plot_tsne()
clean_gpu_memory(model_adacos_v1)
model_adacos_v1 = None
```

--- Setup Model: Arcface ---

Caricamento pesi da:

/kaggle/input/arcface-s50/other/default/1/best\_model\_backbone  
(8).pth...

>>> Pesi caricati con successo (Strict=True).

>>> Estrazione Features in corso...

```
{"model_id": "e607ddf214f2438ead5edc91f2289e35", "version_major": 2, "version_minor": 0}
```

Feature estratte: torch.Size([17534, 512])

===== EVALUATION (5 RUNS) =====

=====

FINAL SUMMARY (Mean  $\pm$  Std Dev over 5 runs)

=====

| METRIC          | MEAN   | STD DEV      | MIN-MAX         |
|-----------------|--------|--------------|-----------------|
| TAR @ FAR=0.01  | 30.64% | $\pm 0.06\%$ | [30.54 - 30.70] |
| TAR @ FAR=0.001 | 11.00% | $\pm 0.08\%$ | [10.89 - 11.10] |
| Rank-1          | 78.67% | $\pm 4.02\%$ | [72.38 - 82.86] |
| Rank-5          | 92.95% | $\pm 1.14\%$ | [91.43 - 94.29] |

===== DISTRIBUTION PLOT (MULTI-FAR) =====

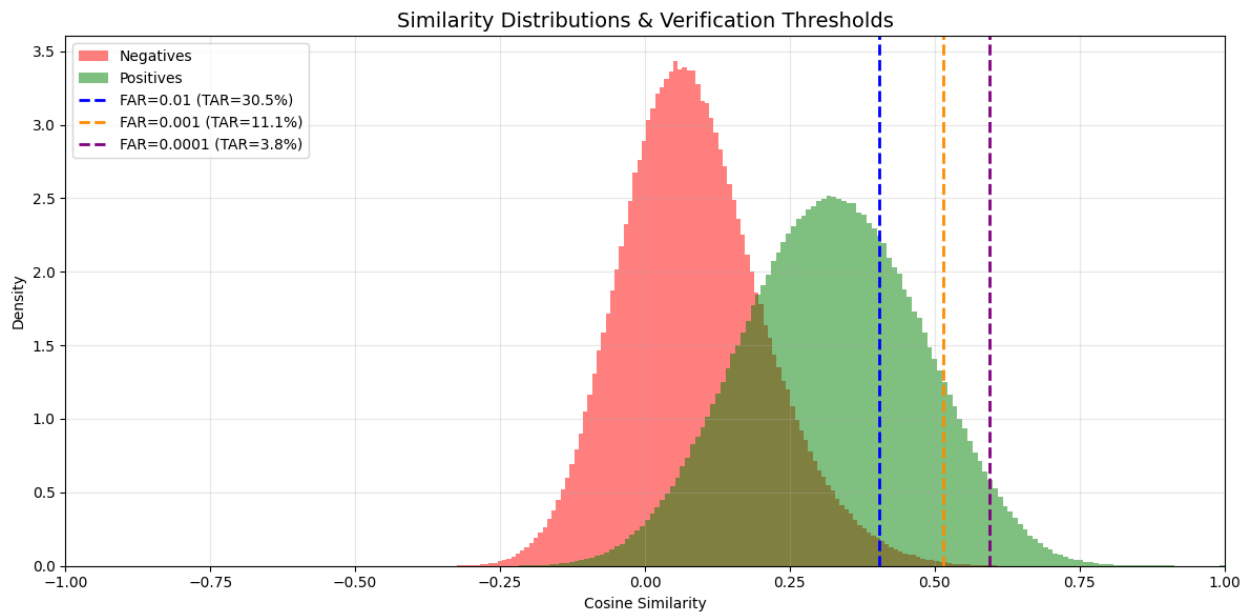
=====

Media Sim Positivi: 0.3239

Media Sim Negativi: 0.0833

-----

| FAR    | TAR    | THRESHOLD |
|--------|--------|-----------|
| 0.01   | 30.52% | 0.4050    |
| 0.001  | 11.12% | 0.5140    |
| 0.0001 | 3.76%  | 0.5940    |



```
===== 3. HARDEST NEGATIVES =====
Pair 1: Sim=0.9215 | ID 74 vs ID 87
Pair 2: Sim=0.7750 | ID 0 vs ID 90
Pair 3: Sim=0.7727 | ID 42 vs ID 95
Pair 4: Sim=0.7640 | ID 14 vs ID 100
Pair 5: Sim=0.7638 | ID 42 vs ID 90
```



## Hardest Negatives

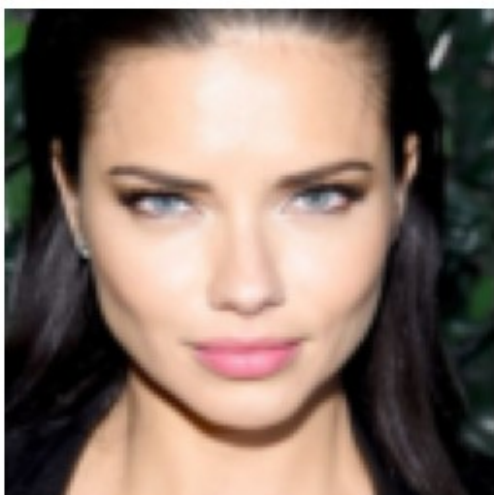
ID: 74



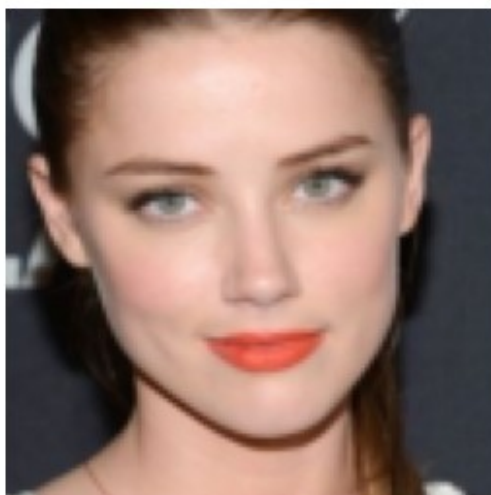
ID: 87



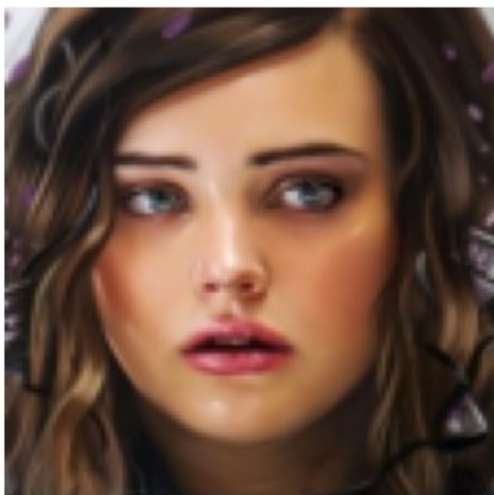
ID: 0



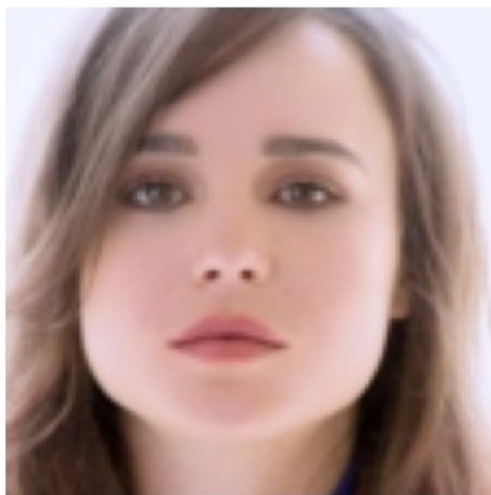
ID: 90



ID: 42



ID: 95



===== 2. IDENTIFICATION (SINGLE RUN) + VIS =====

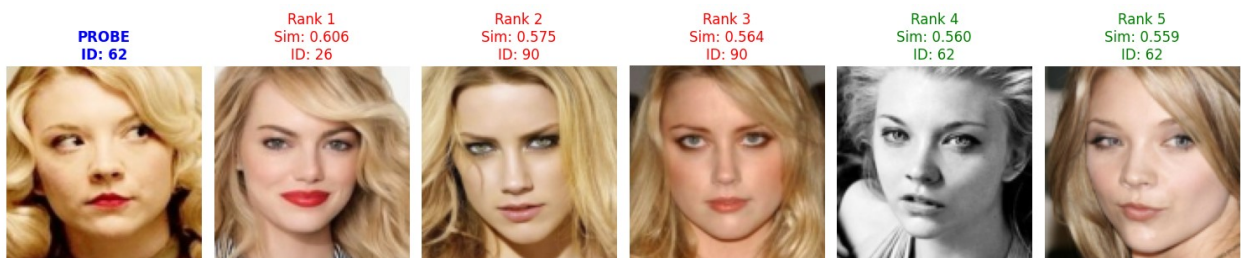
Performance di questa run (per riferimento):

Rank-1: 84.76%

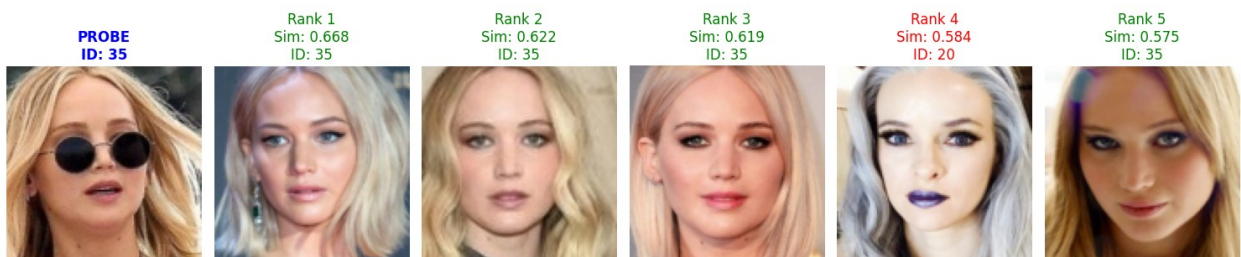
Rank-5: 96.19%

>>> Visualizzazione di 3 esempi di ricerca 1vsN...

Example 1/3 | Probe ID: 62



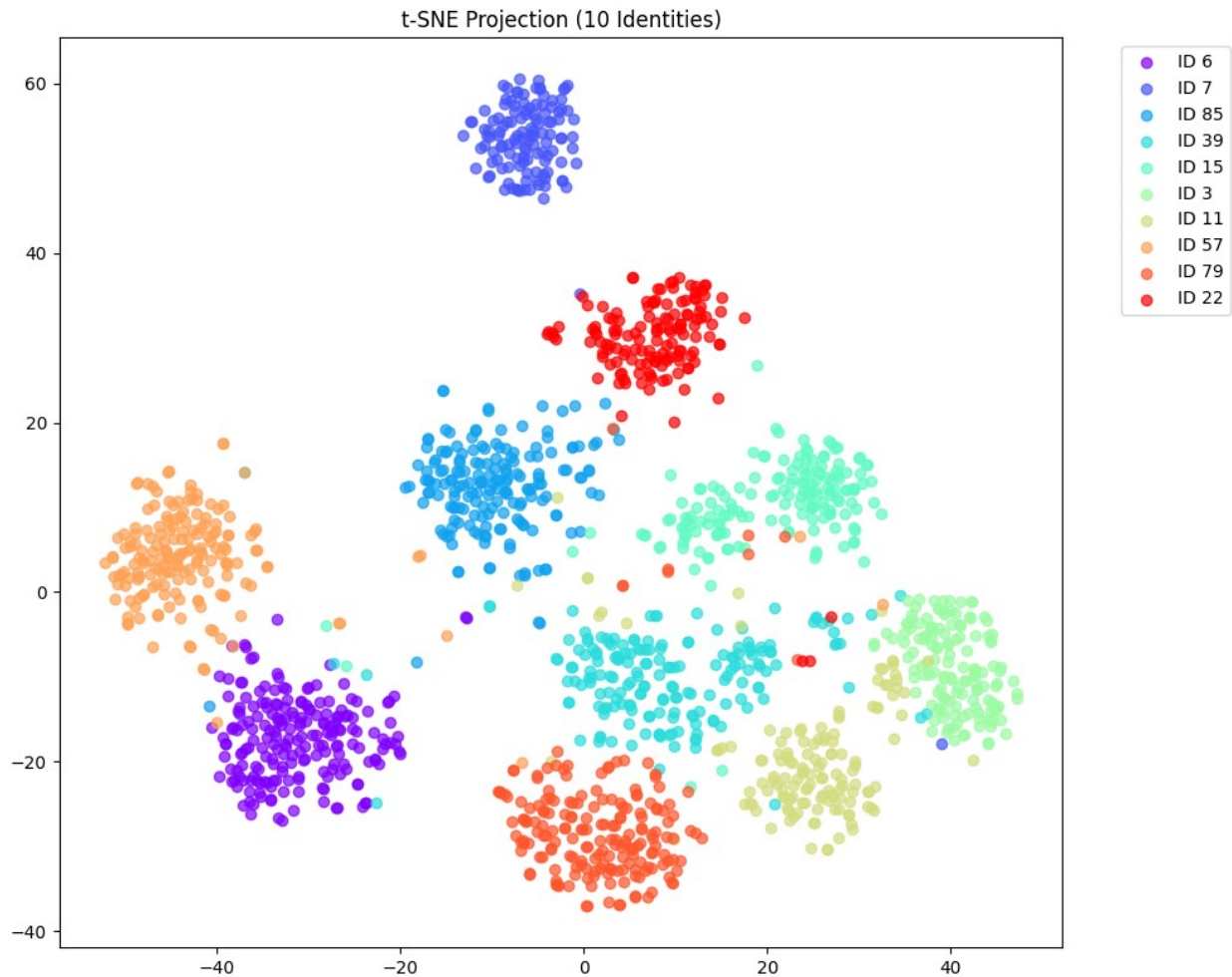
Example 2/3 | Probe ID: 35



Example 3/3 | Probe ID: 57



===== 4. T-SNE VISUALIZATION =====



```
>>> GPU Memory Cleaned.
```

```
@title Test set 1
print("\n--- Setup Model: Arcface ---")
model_adacos_v1 = FaceModel(model_name='seresnet50', use_gdc=False)
path_1 = os.path.join(BASE_PATH, files["arcface_s50"])
load_checkpoint(model_adacos_v1, path_1, device)

evaluator = FaceEvaluator(model_adacos_v1, test_loader_1)
evaluator.run_robust_evaluation(num_runs=5)
evaluator.plot_similarity_distributions()
evaluator.analyze_hardest_negatives()
evaluator.test_identification_lvsN(ranks=[1, 5], num_examples=3)
#evaluator.plot_tsne()
clean_gpu_memory(model_adacos_v1)
model_adacos_v1 = None
```

```
--- Setup Model: Arcface ---
Caricamento pesi da:
```

```
/kaggle/input/arcface-s50/other/default/1/best_model_backbone
(8).pth...
>>> Pesì caricati con successo (Strict=True).
>>> Estrazione Features in corso...

{"model_id":"160be9b6583542018d206ffea52c7c90","version_major":2,"version_minor":0}

Feature estratte: torch.Size([6107, 512])
```

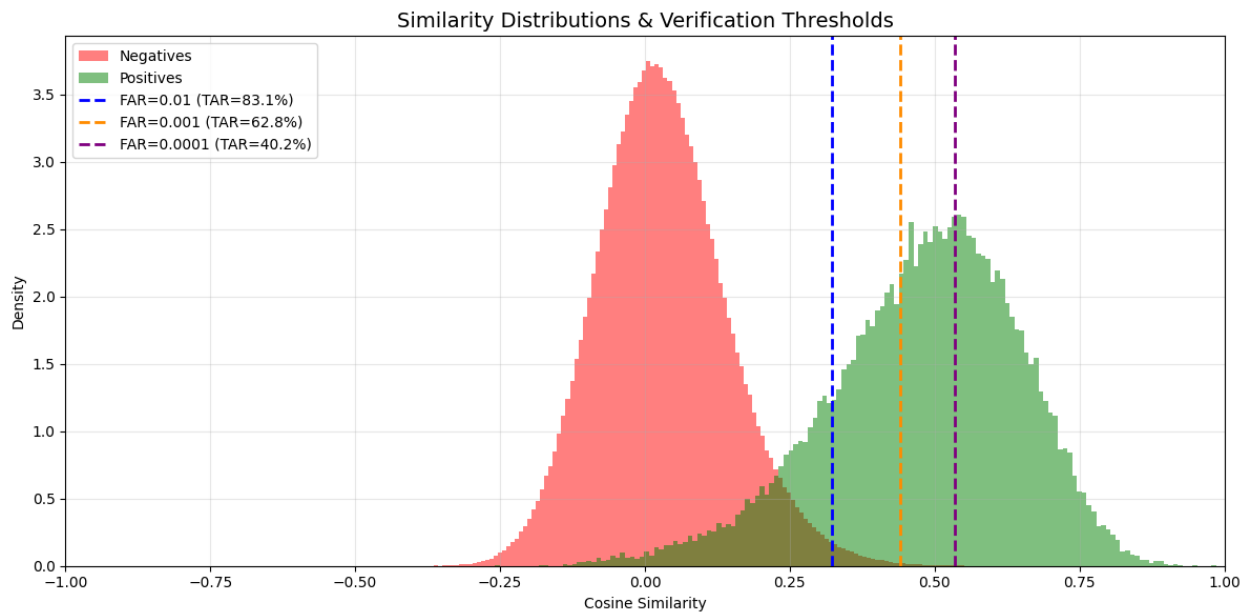
===== EVALUATION (5 RUNS) =====

| FINAL SUMMARY (Mean ± Std Dev over 5 runs) |        |         |                 |
|--------------------------------------------|--------|---------|-----------------|
| METRIC                                     | MEAN   | STD DEV | MIN-MAX         |
| TAR @ FAR=0.01                             | 83.16% | ±0.08%  | [83.04 - 83.30] |
| TAR @ FAR=0.001                            | 62.86% | ±0.56%  | [61.78 - 63.32] |
| Rank-1                                     | 69.34% | ±0.57%  | [68.81 - 70.17] |
| Rank-5                                     | 81.45% | ±0.23%  | [81.04 - 81.65] |

===== DISTRIBUTION PLOT (MULTI-FAR) =====

Media Sim Positivi: 0.4791  
Media Sim Negativi: 0.0304

| FAR    | TAR    | THRESHOLD |
|--------|--------|-----------|
| 0.01   | 83.08% | 0.3228    |
| 0.001  | 62.84% | 0.4404    |
| 0.0001 | 40.25% | 0.5342    |



```
===== 3. HARDEST NEGATIVES =====
Pair 1: Sim=0.8813 | ID 178 vs ID 580
Pair 2: Sim=0.7263 | ID 831 vs ID 1214
Pair 3: Sim=0.7232 | ID 949 vs ID 1317
Pair 4: Sim=0.7197 | ID 167 vs ID 1078
Pair 5: Sim=0.7132 | ID 623 vs ID 1183
```



## Hardest Negatives

ID: 178



ID: 580



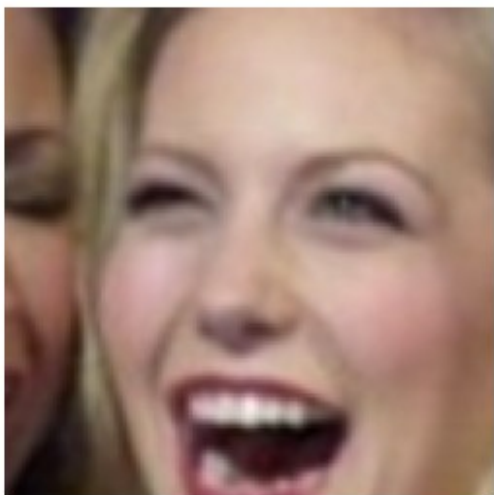
ID: 831



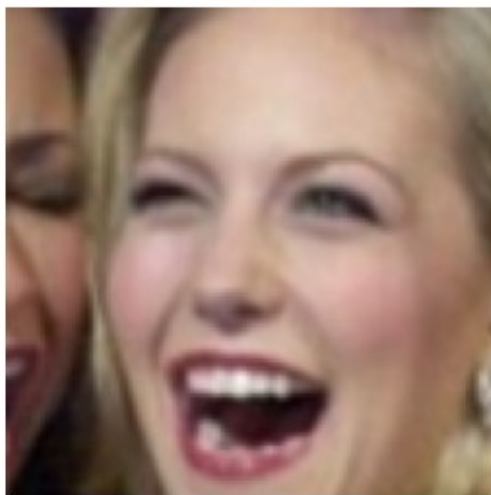
ID: 1214



ID: 949



ID: 1317



===== 2. IDENTIFICATION (SINGLE RUN) + VIS  
=====

Performance di questa run (per riferimento):

Rank-1: 69.94%

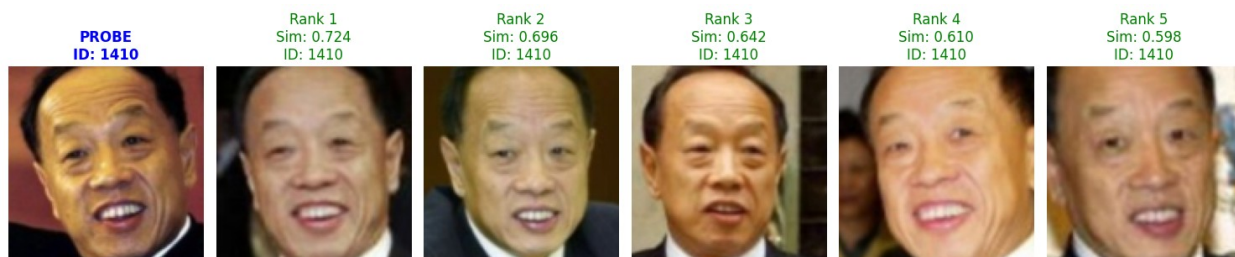
Rank-5: 81.57%

>>> Visualizzazione di 3 esempi di ricerca 1vsN...

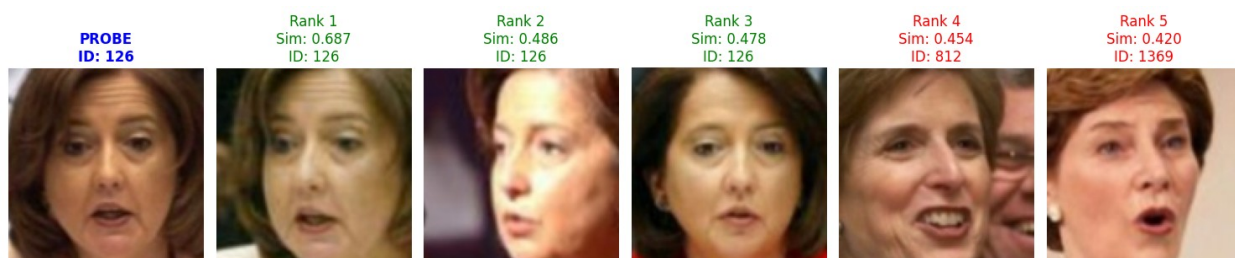
Example 1/3 | Probe ID: 800



Example 2/3 | Probe ID: 1410



Example 3/3 | Probe ID: 126



>>> GPU Memory Cleaned.

```
@title Test set 2
print("\n--- Setup Model: AdaCos V1 ---")
model_adacos_v1 = FaceModel(model_name='seresnet50', use_gdc=False)
path_l = os.path.join(BASE_PATH, files["arcface_s50"])
```



```

load_checkpoint(model_adacos_v1, path_1, device)
evaluator = FaceEvaluator(model_adacos_v1, test_loader_2)
evaluator.run_robust_evaluation(num_runs=5)
evaluator.plot_similarity_distributions()
evaluator.analyze_hardest_negatives()
evaluator.test_identification_lvsN(ranks=[1, 5], num_examples=3)
evaluator.plot_tsne()
clean_gpu_memory(model_adacos_v1)
model_adacos_v1 = None

```

--- Setup Model: AdaCos V1 ---

Caricamento pesi da:

/kaggle/input/arcface-s50/other/default/1/best\_model\_backbone  
(8).pth...

>>> Pesi caricati con successo (Strict=True).

>>> Estrazione Features in corso...

```

{"model_id": "7794177264174acdb3ae1d6c215f90e9", "version_major": 2, "version_minor": 0}

```

Feature estratte: torch.Size([2562, 512])

===== EVALUATION (5 RUNS) =====

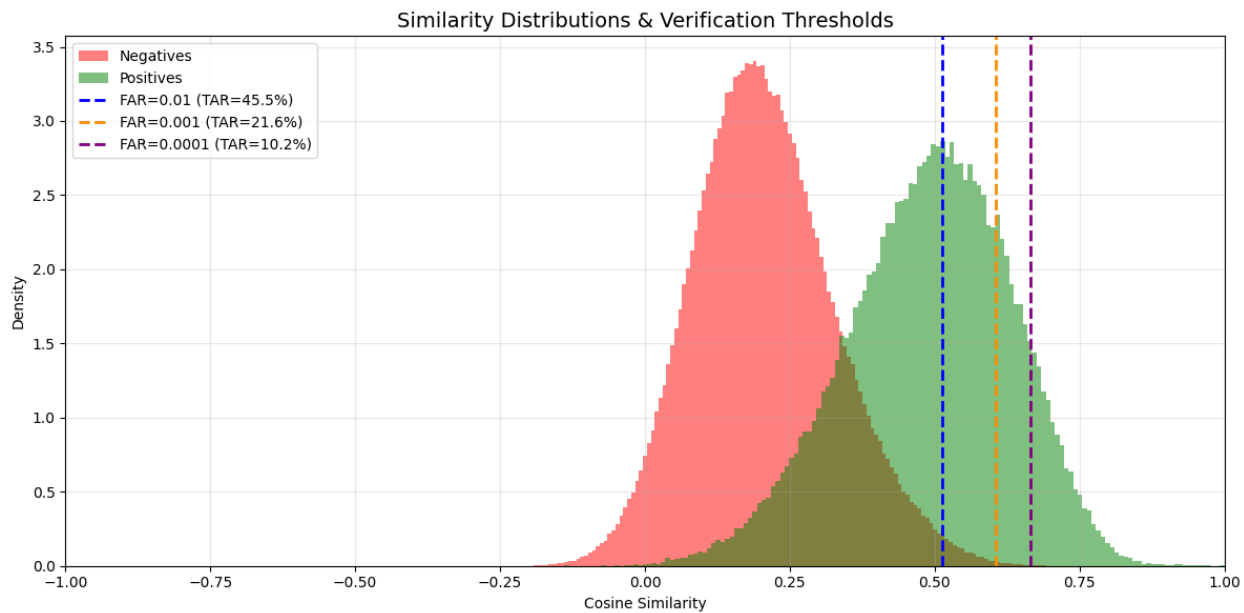
| FINAL SUMMARY (Mean ± Std Dev over 5 runs) |        |         |                  |
|--------------------------------------------|--------|---------|------------------|
| METRIC                                     | MEAN   | STD DEV | MIN-MAX          |
| TAR @ FAR=0.01                             | 45.58% | ±0.09%  | [45.50 - 45.76]  |
| TAR @ FAR=0.001                            | 21.93% | ±0.17%  | [21.64 - 22.16]  |
| Rank-1                                     | 94.19% | ±2.41%  | [90.32 - 96.77]  |
| Rank-5                                     | 98.71% | ±1.58%  | [96.77 - 100.00] |

===== DISTRIBUTION PLOT (MULTI-FAR) =====

Media Sim Positivi: 0.4894

Media Sim Negativi: 0.2063

| FAR    | TAR    | THRESHOLD |
|--------|--------|-----------|
| 0.01   | 45.50% | 0.5138    |
| 0.001  | 21.61% | 0.6053    |
| 0.0001 | 10.16% | 0.6650    |



```
===== 3. HARDEST NEGATIVES =====
Pair 1: Sim=0.7624 | ID 1440 vs ID 1436
Pair 2: Sim=0.7618 | ID 1436 vs ID 1440
Pair 3: Sim=0.7574 | ID 1443 vs ID 1455
Pair 4: Sim=0.7559 | ID 1436 vs ID 1440
Pair 5: Sim=0.7549 | ID 1447 vs ID 1431
```

## Hardest Negatives

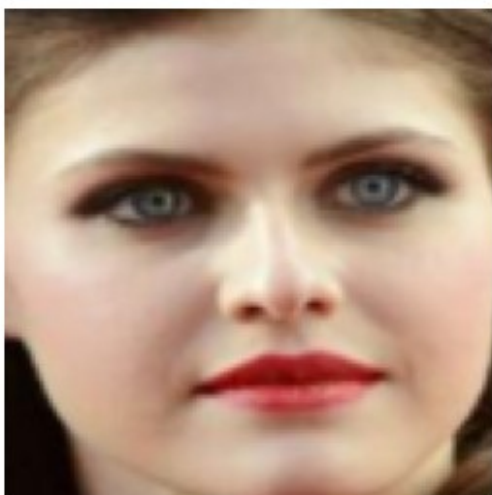
ID: 1440



ID: 1436



ID: 1436



ID: 1440



ID: 1443



ID: 1455



===== 2. IDENTIFICATION (SINGLE RUN) + VIS  
=====

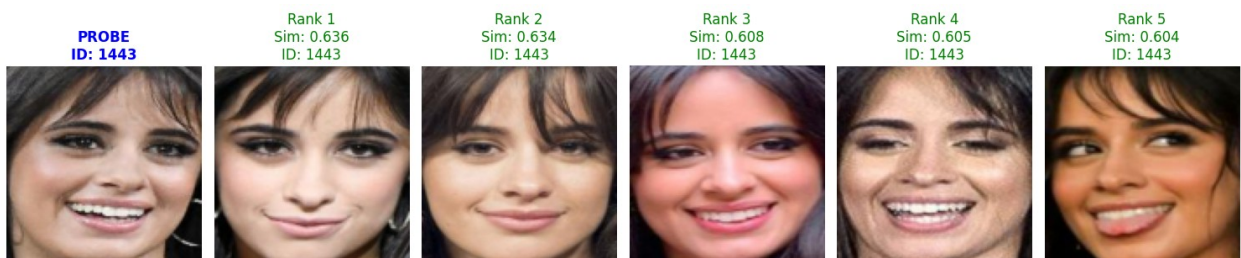
Performance di questa run (per riferimento):

Rank-1: 100.00%

Rank-5: 100.00%

>>> Visualizzazione di 3 esempi di ricerca 1vsN...

Example 1/3 | Probe ID: 1443



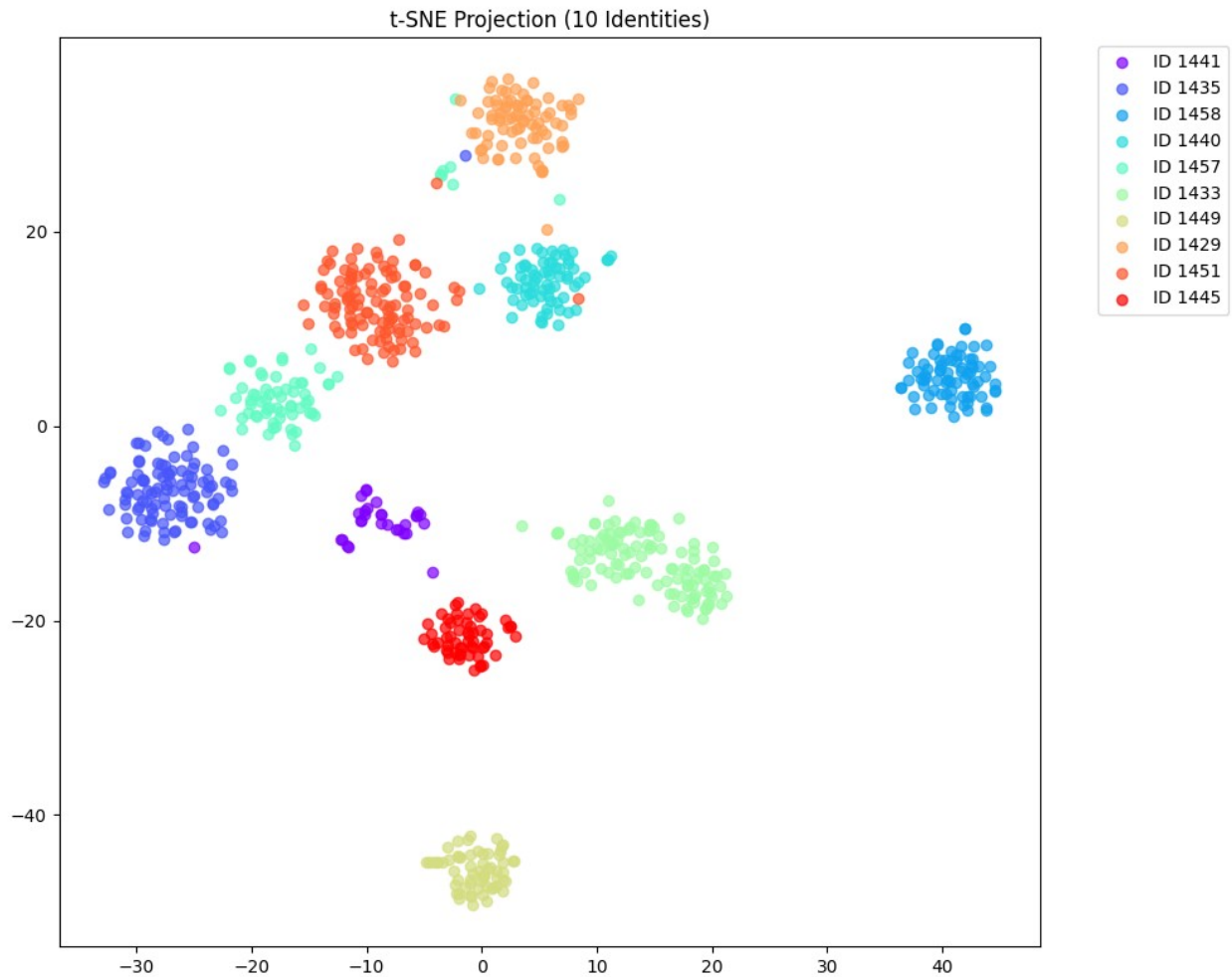
Example 2/3 | Probe ID: 1433



Example 3/3 | Probe ID: 1458



===== 4. T-SNE VISUALIZATION =====



```
>>> GPU Memory Cleaned.
```

## LOSSES AND TRAINING LOOP

```
ROOT_DIR = "/kaggle/input/vggface2-112x112/vggface2_112x112"

image_paths = []
labels = []

Prendiamo SOLO le cartelle id_xxx
id_folders = sorted([
 d for d in os.listdir(ROOT_DIR)
 if os.path.isdir(os.path.join(ROOT_DIR, d))
])

Barra 1: progresso sulle cartelle
for id_folder in tqdm(id_folders, desc="Cartelle (identità)",
 position=0, mininterval=1.0):
```

```

folder_path = os.path.join(ROOT_DIR, id_folder)

images = [
 f for f in os.listdir(folder_path)
 if f.lower().endswith((".jpg", ".png", ".jpeg"))
]

Barra 2: immagini nella cartella corrente
for img_name in tqdm(
 images,
 desc=f"Immagini in {id_folder}",
 position=1,
 leave=False
):
 full_path = os.path.join(folder_path, img_name)

 image_paths.append(full_path)
 labels.append(id_folder)

Costruzione DataFrame
df_train= pd.DataFrame({
 "image_path": image_paths,
 "label": labels
})

print(df_train.head())
print("Numero totale immagini:", len(df_train))
print("Numero classi:", df_train["label"].nunique())
import numpy as np
import pandas as pd
from tqdm.auto import tqdm

import numpy as np
import pandas as pd

def sample_classes_and_images_fast(df, n_classes=None,
n_images_per_class=None, seed=42):
 rng = np.random.default_rng(seed)

 # 1) Scegli le classi
 classes = df["label"].unique()

 if n_classes is not None:
 if n_classes > len(classes):
 raise ValueError("n_classes > numero classi disponibili")
 chosen_classes = rng.choice(classes, size=n_classes,
replace=False)
 else:
 chosen_classes = classes

```

```

2) Filtra solo le classi scelte
df = df[df["label"].isin(chosen_classes)]

3) Se non limiti immagini → fine
if n_images_per_class is None:
 return df.reset_index(drop=True)

4) Campionamento vettorizzato per gruppo
df_out = (
 df
 .groupby("label", group_keys=False)
 .apply(
 lambda x: x.sample(
 n=min(len(x), n_images_per_class),
 random_state=seed
)
)
 .reset_index(drop=True)
)

return df_out

=====
ESEMPIO DI UTILIZZO
=====

df_train_small = sample_classes_and_images_fast(
 df_train,
 n_classes=3000,
 n_images_per_class=80,
 seed=42
)

print("Numero immagini:", len(df_train_small))
print("Numero classi:", df_train_small["label"].nunique())
print(df_train_small.head())

print("Numero immagini:", len(df_train_small))
print("Numero classi:", df_train_small["label"].nunique())
print(df_train_small.head())

=====
1. IMPLEMENTAZIONI CUSTOM DELLE LOSS (Invariate)
=====
class ArcFace(nn.Module):
 def __init__(self, in_features, out_features, s=64.0, m=0.50):
 super(ArcFace, self).__init__()

```



```

 self.in_features = in_features
 self.out_features = out_features
 self.s = s
 self.m = m
 self.weight = Parameter(torch.FloatTensor(out_features,
in_features))
 nn.init.xavier_uniform_(self.weight)
 self.cos_m = math.cos(m)
 self.sin_m = math.sin(m)
 self.th = math.cos(math.pi - m)
 self.mm = math.sin(math.pi - m) * m

 def forward(self, input, label):
 cosine = F.linear(F.normalize(input),
F.normalize(self.weight))
 sine = torch.sqrt((1.0 - torch.pow(cosine, 2)).clamp(0, 1))
 phi = cosine * self.cos_m - sine * self.sin_m
 phi = torch.where(cosine > self.th, phi, cosine - self.mm)
 one_hot = torch.zeros(cosine.size(), device=input.device)
 one_hot.scatter_(1, label.view(-1, 1).long(), 1)
 output = (one_hot * phi) + ((1.0 - one_hot) * cosine)
 output *= self.s
 return output

class CosFace(nn.Module):
 def __init__(self, in_features, out_features, s=64.0, m=0.35):
 super(CosFace, self).__init__()
 self.in_features = in_features
 self.out_features = out_features
 self.s = s
 self.m = m
 self.weight = Parameter(torch.FloatTensor(out_features,
in_features))
 nn.init.xavier_uniform_(self.weight)

 def forward(self, input, label):
 cosine = F.linear(F.normalize(input),
F.normalize(self.weight))
 phi = cosine - self.m
 one_hot = torch.zeros(cosine.size(), device=input.device)
 one_hot.scatter_(1, label.view(-1, 1).long(), 1)
 output = (one_hot * phi) + ((1.0 - one_hot) * cosine)
 output *= self.s
 return output

class AdaCos(nn.Module):
 def __init__(self, in_features, out_features):
 super(AdaCos, self).__init__()
 self.in_features = in_features
 self.out_features = out_features

```

```

 self.s = math.sqrt(2) * math.log(out_features - 1)
 self.m = m
 self.weight = Parameter(torch.FloatTensor(out_features,
in_features))
 nn.init.xavier_uniform_(self.weight)

 def forward(self, input, label=None):
 cosine = F.linear(F.normalize(input),
F.normalize(self.weight))
 if label is None: return cosine
 theta = torch.acos(torch.clamp(cosine, -1.0 + 1e-7, 1.0 - 1e-
7))
 one_hot = torch.zeros_like(cosine)
 one_hot.scatter_(1, label.view(-1, 1).long(), 1)
 with torch.no_grad():
 B_avg = torch.where(one_hot < 1, torch.exp(self.s *
cosine), torch.zeros_like(cosine))
 B_avg = torch.sum(B_avg) / input.size(0)
 theta_med = torch.median(theta[one_hot == 1])
 self.s = torch.log(B_avg) / torch.cos(torch.min(math.pi/4
* torch.ones_like(theta_med), theta_med))
 output = self.s * cosine
 return output

=====
2. UTILS METRICHE
=====
def evaluate_tar_far(pos_sims, neg_sims, far_target=1e-3):
 neg_sims = np.sort(neg_sims)
 pos_sims = np.sort(pos_sims)
 num_neg = len(neg_sims)
 cutoff_idx = int((1.0 - far_target) * num_neg)
 cutoff_idx = min(cutoff_idx, num_neg - 1)
 threshold = neg_sims[cutoff_idx]
 num_pos = len(pos_sims)
 count_below_thresh = np.searchsorted(pos_sims, threshold)
 count_above_thresh = num_pos - count_below_thresh
 tar = count_above_thresh / num_pos
 return tar, threshold

=====
3. CLASSE TRAINER AGGIORNATA
=====
class FaceTrainer:
 def __init__(self, model, train_loader, val_loader,
 loss_type='arcface', num_classes=None,
embedding_size=512,
 device='cuda', lr=0.001, patience=2):

 self.model = model.to(device)

```

```

self.train_loader = train_loader
self.val_loader = val_loader
self.device = device
self.loss_type = loss_type.lower()
self.patience = patience

--- Configurazione Loss ---
if self.loss_type == 'arcface':
 self.metric_head = ArcFace(embedding_size, num_classes,
s=30.0, m=0.3).to(device)
elif self.loss_type == 'cosface':
 self.metric_head = CosFace(embedding_size, num_classes,
s=30.0, m=0.45).to(device)
elif self.loss_type == 'adacos':
 self.metric_head = AdaCos(embedding_size,
num_classes).to(device)
else:
 raise ValueError("loss_type non valido")

--- Optimizer ---
self.optimizer = optim.AdamW([
 {'params': self.model.parameters()},
 {'params': self.metric_head.parameters()}
], lr=lr, weight_decay=0.01)

self.criterion = nn.CrossEntropyLoss()

--- SCHEDULER ---
self.scheduler = optim.lr_scheduler.ReduceLROnPlateau(
 self.optimizer, mode='max', factor=0.5, patience=0
)

def train_epoch(self, epoch_idx):
 self.model.train()
 self.metric_head.train()

 pbar = tqdm(self.train_loader, desc=f"Train Ep {epoch_idx}")
 running_loss = 0.0
 running_corrects = 0
 batch_angles = []

 for imgs, labels in pbar:
 imgs, labels = imgs.to(self.device),
labels.to(self.device)
 self.optimizer.zero_grad()

 embeddings = self.model(imgs)
 logits_margin = self.metric_head(embeddings, labels)
 loss = self.criterion(logits_margin, labels)

```

```

 loss.backward()
 self.optimizer.step()

 running_loss += loss.item() * imgs.size(0)

 with torch.no_grad():
 e_norm = F.normalize(embeddings, p=2, dim=1)
 w_norm = F.normalize(self.metric_head.weight, p=2,
dim=1)

 logits_clean = torch.mm(e_norm, w_norm.t())
 preds = torch.argmax(logits_clean, dim=1)
 running_corrects += torch.sum(preds == labels.data)

 target_W = w_norm[labels, :]
 cos_theta = (e_norm * target_W).sum(dim=1).clamp(-1,
1)

 batch_angles.append((torch.acos(cos_theta) * 180 /
np.pi).mean().item()))

 avg_angle = np.mean(batch_angles[-20:])
 pbar.set_postfix({'Loss': f"{loss.item():.4f}", 'Acc':
f"{torch.sum(preds==labels)/len(labels):.2f}", 'Avgθ':
f"{avg_angle:.1f}°"})

 return running_loss / len(self.train_loader.dataset)

def validate_and_plot(self, epoch_idx):
 self.model.eval()
 print(f"Validating Ep {epoch_idx}...")

 all_embeddings, all_labels = [], []

 with torch.no_grad():
 for imgs, labels in self.val_loader:
 imgs = imgs.to(self.device)
 emb = F.normalize(self.model(imgs), p=2, dim=1)
 all_embeddings.append(emb.cpu())
 all_labels.append(labels.cpu())

 all_embeddings = torch.cat(all_embeddings)
 all_labels = torch.cat(all_labels)

 sim_matrix = torch.mm(all_embeddings, all_embeddings.t())

 labels_expand = all_labels.view(-1, 1)
 mask_pos = torch.eq(labels_expand, labels_expand.t())

 diag_mask = torch.eye(mask_pos.size(0), dtype=torch.bool)
 mask_pos = mask_pos & (~diag_mask)
 mask_neg = ~mask_pos & (~diag_mask)

```

```

pos_sims = sim_matrix[mask_pos].numpy()
neg_sims = sim_matrix[mask_neg].numpy()
if len(neg_sims) > 1_000_000:
 neg_sims = np.random.choice(neg_sims, 1_000_000,
replace=False)

far_target = 1e-3
val_tar, threshold = evaluate_tar_far(pos_sims, neg_sims,
far_target=far_target)

print(f"\n>>> VALIDATION METRICS (Ep {epoch_idx}) <<<")
print(f" TAR @ FAR={far_target}: {val_tar*100:.2f}%")
print(f" Threshold ottimale: {threshold:.4f}")
print(f" Avg Pos Sim: {np.mean(pos_sims):.3f} | Avg Neg
Sim: {np.mean(neg_sims):.3f}")

plt.figure(figsize=(10, 5))
plt.hist(neg_sims, bins=100, alpha=0.5, color='red',
label='Negatives', density=True)
plt.hist(pos_sims, bins=100, alpha=0.5, color='green',
label='Positives', density=True)
plt.axvline(x=threshold, color='blue', linestyle='--',
label=f'Tresh @ FAR {far_target}')

plt.title(f"Ep {epoch_idx} - TAR={val_tar*100:.2f}% @
FAR={far_target}")
plt.legend()
plt.show()

return val_tar

def run(self, epochs=100):
 best_tar = 0.0
 patience_counter = 0

 # --- 1. PRINT PARAMETRI LOSS INIZIALI ---
 print("\n" + "="*40)
 print(f" STARTING TRAINING - Loss: {self.loss_type.upper()}")

 # Recupero i parametri dalla classe della Loss
 # Nota: AdaCos ha 's' dinamico, quindi stampiamo quello
iniziale
 current_s = self.metric_head.s
 if isinstance(current_s, torch.Tensor):
 current_s = current_s.item()

 current_m = getattr(self.metric_head, 'm', 'Dynamic') # AdaCos
ha m nel costruttore

```

```

print(f" Initial Scale (s): {current_s:.2f}")
print(f" Margin (m): {current_m}")
print("="*40 + "\n")

for epoch in range(1, epochs + 1):
 train_loss = self.train_epoch(epoch)

 # Validazione
 current_tar = self.validate_and_plot(epoch)

 # Scheduler
 self.scheduler.step(current_tar)

 # --- 2. SALVATAGGIO MODELLO (BEST) ---
 if current_tar > best_tar:
 best_tar = current_tar
 patience_counter = 0

 # Nome file univoco per Kaggle
 save_path = f"best_model_backbone{epoch}.pth"

 # Salviamo solo la backbone (self.model)
 torch.save(self.model.state_dict(), save_path)

 print(f"*** New Best Model! TAR: {best_tar*100:.2f}% -
> Saved to {save_path} ***")
 else:
 patience_counter += 1
 print(f"Early Stopping Counter:
{patience_counter}/{self.patience}")

 if patience_counter >= self.patience:
 print("\n===== EARLY STOPPING TRIGGERED
=====")
 print(f"Il modello non migliora da {self.patience}
epoche.")
 print(f"Best TAR: {best_tar*100:.2f}%")
 break

=====
ESECUZIONE
=====

1. Crea il modello (Assicurati che FaceModel sia definito come nel
tuo snippet)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = FaceModel(model_name='seresnet50', embedding_size=512,
pretrained=True).to(device)

2. Inizializza il Trainer

```

```
Puoi cambiare loss_type in: 'arcface', 'cosface', 'adacos'
trainer = FaceTrainer(
 model=model,
 train_loader=train_loader,
 val_loader=val_loader,
 loss_type='adacos', # <--- Seleziona qui la loss
 num_classes=num_classes_train,
 embedding_size=512,
 device=device,
 lr=0.1 # LR standard per fine-tuning backbone pre-allenata
)

3. Start
trainer.run(epochs=15)
```